

Universidade do Minho

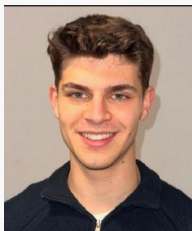
Licenciatura em Engenharia Informática

Programação Orientada a Objetos - 2025/2026

POO

Trabalho Prático

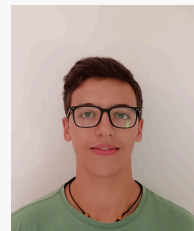
Grupo PL53



Afonso Barros
N.º a112178



Matheus Azevedo
N.º a111430



Martim Monteiro
N.º a111013

Braga, Maio de 2026

Resumo

Este relatório documenta o desenvolvimento do DomusControl, uma aplicação de automação doméstica desenvolvida no âmbito da Unidade Curricular de Programação Orientada a Objetos do ano letivo 2025/2026. A aplicação permite criar e gerir casas com as suas divisões e dispositivos, interagir individualmente com cada dispositivo, e definir automações, escalonamentos e cenários que atuam sobre múltiplos dispositivos em simultâneo. Para além disso, o sistema aprende com o histórico de interações manuais do utilizador e sugere automações e escalonamentos de forma automática.

A aplicação foi construída com uma arquitetura em camadas que separa a lógica de negócio e a interface com o utilizador em modo texto. Foram aplicados padrões de desenho como o Facade, e a extensibilidade foi garantida através de uma hierarquia de classes e interfaces de capacidade que permite adicionar novos tipos de dispositivos sem alterar código existente. A simulação ambiental - temperatura, luminosidade e condição meteorológica - é integrada com sensores físicos que alimentam condições de automação. O estado da aplicação é persistido em ficheiro binário por serialização Java.

Todos os requisitos propostos foram implementados, cobrindo os quatro patamares definidos no enunciado, incluindo o nível mais avançado de geração automática de sugestões com base no histórico de uso.

Palavras-chave: automação doméstica, Java, programação orientada a objetos, Facade, simulação ambiental, sensores, serialização, MVC.

Índice

1. Introdução	5
1.1. Organização dos Requisitos	5
1.2. Estratégia de Desenvolvimento	5
1.3. Padrões de Desenho	6
1.3.1. Facade	6
1.3.2. Hierarquia e Capacidades	6
2. Arquitetura de Classes	7
2.1. Pacote <code>devices</code>	7
2.1.1. Classe <code>Device</code>	7
2.1.2. Hierarquia de Tipos	8
2.1.3. Classes Concretas de Dispositivos	8
2.1.4. Subpacote <code>devices.sensors</code>	9
2.2. Pacote <code>simulation</code>	10
2.3. Pacote <code>houses</code>	11
2.3.1. Classe <code>House</code>	11
2.3.2. Classe <code>HouseManager</code>	12
2.3.3. Classe <code>DivisionInfo</code>	13
2.4. Pacote <code>routines</code>	14
2.4.1. Classe <code>Routine</code>	14
2.4.2. Classe <code>Scenario</code>	14
2.4.3. Classe <code>Automation</code>	15
2.4.4. Interface <code>Action</code> e Implementações	15
2.4.5. Interface <code>Condition</code> e Implementações	16
2.4.6. Classe <code>RoutineManager</code>	18
2.5. Pacote <code>suggestions</code>	19
2.5.1. <code>InteractionType</code>	19
2.5.2. Classe <code>DeviceInteraction</code>	19
2.5.3. <code>InteractionLogger</code>	20
2.5.4. Classe <code>SuggestionEngine</code>	20
2.5.5. <code>AutomationSuggestion</code>	22
2.6. Pacote <code>user</code>	22
2.6.1. Enum <code>UserRole</code>	23
2.6.2. Classe <code>User</code>	23
2.6.3. Classe <code>UserManager</code>	23
2.7. Pacote <code>exceptions</code>	23
2.8. Camada de Interface	24
2.9. Classe <code>Menu</code>	24
2.9.1. Classe <code>DomusControlUI</code>	24
2.9.2. Classe <code>UserUI</code>	25
2.9.3. Classe <code>HouseUI</code>	26
2.9.4. Classe <code>ActionsUI</code>	27
3. Implementação	28

3.1. Gestão do Tempo e Ambiente de Simulação	28
3.2. Consistência de Referências em House	28
3.3. Serialização e Referências Partilhadas	28
3.4. Registo de Interações para Sugestões	29
3.5. Encapsulamento e Cópias Defensivas	29
3.6. Estatísticas	29
4. Arquitetura MVC Delegate	31
4.1. Conceito	31
4.1.1. Conceito	31
4.2. Implementação no DomusControl	31
4.2.1. Fluxo de uma Operação Típica	32
5. Utilização da Aplicação	33
5.1. Compilação e Execução	33
5.2. Ecrã de Boas-Vindas	33
5.3. Registo e Login	34
5.4. Menu Principal do Utilizador	34
5.4.1. Criar uma Casa	35
5.4.2. Avançar o Tempo	35
5.4.3. Estatísticas	36
5.5. Menu de uma Casa	36
5.5.1. Adicionar Dispositivo	37
5.5.2. Operar um Dispositivo	38
5.5.3. Criar uma Automação	39
5.5.4. Ver Sugestões	39
5.6. Gravar e Carregar Estado	40
6. Testes	40
7. Conclusão	42

1. Introdução

O DomusControl é uma aplicação de simulação de um sistema de automação doméstica, cujo o objetivo é permitir que utilizadores criem casas virtuais, organizem os seus dispositivos conectados por divisões, e definam regras automáticas de funcionamento - tudo num ambiente de simulação onde o tempo e as condições ambientais podem ser controlados explicitamente.

Do ponto de vista da Unidade Curricular, o projeto serve como campo de aplicação dos conceitos de Programação Orientada a Objetos: encapsulamento, herança, polimorfismo, interfaces, padrões de desenho e persistência. Este relatório descreve as decisões de arquitetura tomadas, a estrutura de classes resultante, e o funcionamento da aplicação final.

1.1. Organização dos Requisitos

O enunciado define quatro patamares de requisitos com notas máximas crescentes. A análise cuidadosa desses patamares permitiu identificar três pontos fundamentais em torno dos quais o sistema se organiza.

Dispositivos: O sistema tem de suportar diferentes tipos de dispositivos com comportamentos distintos: ligar e desligar, regular intensidade, controlar o grau de abertura, e ajustar a cor. O enunciado é explícito ao exigir que novos tipos de dispositivos possam ser adicionados criando novas classes, sem obrigar a alterar o código da classe computacional principal.

Utilizadores e casas: Cada utilizador pode ter acesso a várias casas com papéis distintos: administrador, que gere a instalação, ou utilizador, que apenas opera os dispositivos. As casas agrupam dispositivos em divisões e são a unidade organizacional central do sistema.

Rotinas - Automações, Escalonamentos e Cenários: Automações e escalonamentos combinam condições com ações: quando determinada condição é verificada, um conjunto de ações é executado automaticamente. Cenários são sequências de ações que o utilizador pode disparar manualmente. O nível mais avançado exige que o sistema aprenda com o padrão de uso do utilizador e proponha novas automações.

As decisões de arquitetura tomadas tiveram em conta estas condições.

1.2. Estratégia de Desenvolvimento

O desenvolvimento foi organizado de forma incremental, seguindo os quatro patamares do enunciado como marcos naturais. A equipa começou por definir a hierarquia de classes e interfaces antes de avançar para a implementação concreta, o que permitiu tomar as decisões de encapsulamento de forma mais informada e evitar refactorizações tardias.

Como sistema de build foi utilizado o Gradle. O código está documentado com Javadoc em todas as classes, métodos e parâmetros. A organização em pacotes segue a divisão lógica do domínio, separando claramente dispositivos, sensores, simulação, casas, rotinas, sugestões, utilizadores e interface.

Para verificação funcional do sistema foi desenvolvido um estado de teste pré-carregado com casas, divisões, dispositivos, utilizadores e rotinas, permitindo testar as funcionalidades sem necessidade de criar tudo manualmente a cada execução.

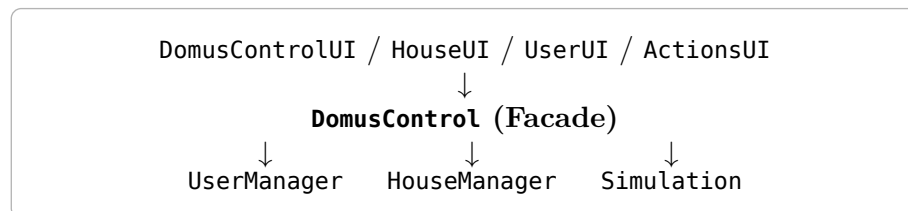
1.3. Padrões de Desenho

A análise dos requisitos levou à identificação e aplicação de padrões de desenho de forma a responder às necessidades da aplicação.

1.3.1. Facade

O padrão **Facade** foi aplicado na classe **DomusControl**, que funciona como a porta de acesso ao modelo a partir da camada de interface. A UI acede aos métodos de **DomusControl** para interagir com o modelo.

Este padrão simplifica a camada de UI, isola-a das estruturas internas do modelo, e facilita a evolução futura: alterar a forma como os dados são armazenados internamente não requer alterações na UI enquanto a interface da fachada se mantiver estável.



1.3.2. Hierarquia e Capacidades

A hierarquia de dispositivos assenta em classes abstractas que aproveitam código comum entre dispositivos do mesmo tipo: **SwitchableDevice** estende **Device**, **AdjustableDevice** estende **SwitchableDevice**, e **OpenableDevice** estende **Device**. Esta estrutura permite reutilizar a lógica de **turnOn**, **turnOff** e **isOn** sem duplicação.

A excepção é **ColorAdjustableDevice**, que é definida como interface em vez de classe abstracta. A razão é que **Lamp** precisa de herdar de **AdjustableDevice** (para o brilho) e simultaneamente suportar temperatura de cor, no entanto, Java não permite herança múltipla de classes. A interface resolve o problema sem comprometer a hierarquia existente. Qualquer dispositivo futuro que suporte temperatura de cor basta implementar a interface, independentemente da sua posição na hierarquia de herança.

2. Arquitetura de Classes

Esta secção descreve a arquitetura de classes da aplicação, organizada pelos pacotes principais. O diagrama de classes completo é fornecido como ficheiro externo em anexo.

2.1. Pacote devices

2.1.1. Classe Device

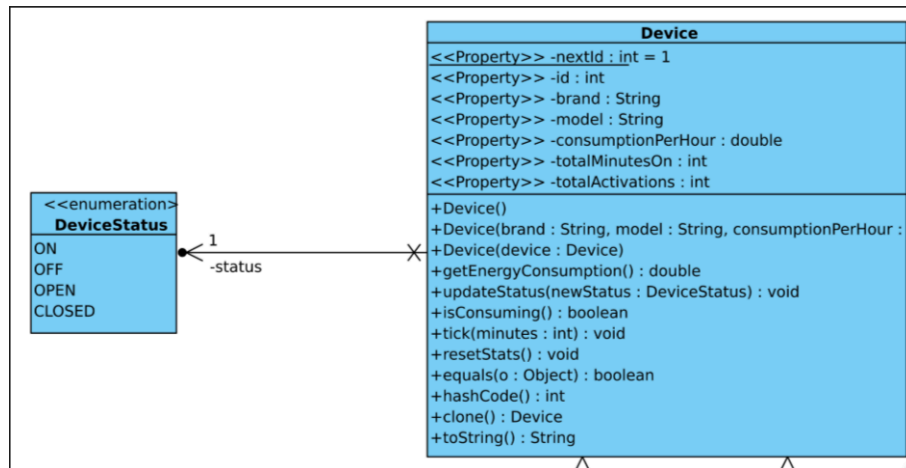


Figura 1: Diagrama de classes da classe Device

Device é a classe abstracta que constitui a raiz da hierarquia de dispositivos. Define os atributos comuns a todos os dispositivos:

- **id** - identificador único auto-incrementado por um contador estático **nextId**. O método estático **setNextId(int lastId)** permite repor o contador ao carregar um estado de ficheiro, garantindo que novos dispositivos não colidem com IDs já existentes.
- **brand** e **model** - marca e modelo do dispositivo.
- **consumptionPerHour** - consumo em Wh/h, usado para calcular o consumo total da casa.
- **status** - estado atual do dispositivo, representado pelo enum **DeviceStatus** (ON, OFF, OPEN, CLOSED).
- **totalMinutesOn** - número total de minutos em que o dispositivo esteve ligado, acumulado pelo método **tick(int minutes)** chamado em cada avanço de tempo.
- **totalActivations** - número total de vezes que o dispositivo foi ligado, incrementado em cada passagem de **OFF** para **ON** ou se **CLOSED** para **OPEN**.

Estes dois últimos campos alimentam diretamente as consultas estatísticas do requisito 8.2: *top 3 dispositivos por tempo de utilização* e *top 3 dispositivos por número de ativações*.

2.1.2. Hierarquia de Tipos

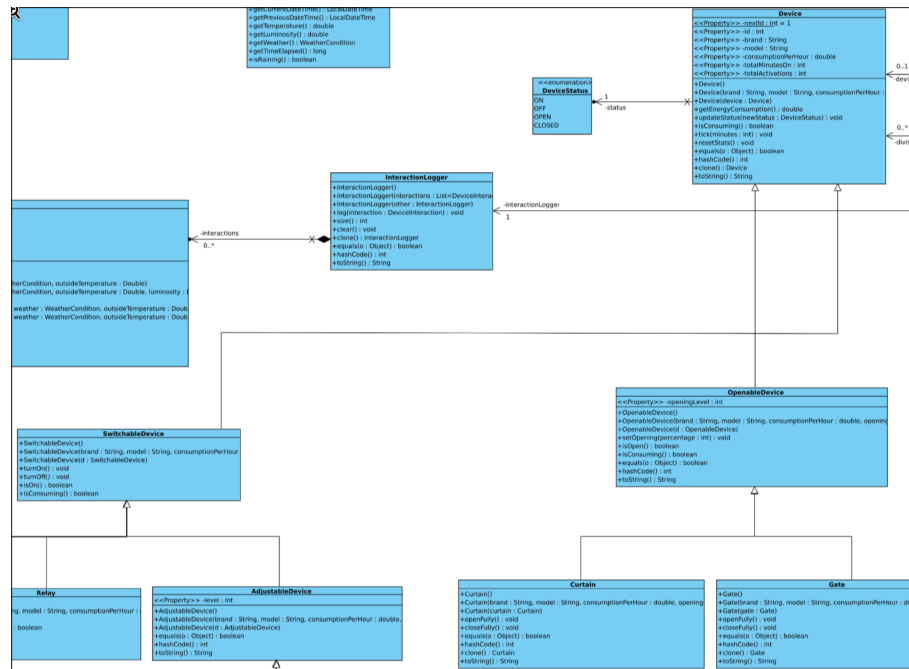


Figura 2: Diagrama de classes com a hierarquia de Devices

Os dispositivos são organizadas numa hierarquia de classes abstractas e uma interface:

SwitchableDevice estende Device e implementa os métodos turnOn(), turnOff() e isOn().

`AdjustableDevice` estende `SwitchableDevice` e adiciona um campo `level` (0–100) com comportamento definido de acordo o respetivo dispositivo que o implementa. A classe `SwitchableDevice` por si não desliga ou liga dispositivos de acordo com este campo, deixando para as classes que a implementam a possibilidade de dar *Override* nesse método. Por exemplo uma lampada se for colocada a uma luminosidade de 0 é automaticamente desligada; enquanto que se colocar o volume de uma coluna a 0 não é desligada.

`OpenableDevice` estende `Device` e adiciona `openingLevel` (0–100), onde 0 representa totalmente fechado e 100 totalmente aberto. Implementa `setOpening(int)`, `getOpeningLevel()` e o método `isOpen()`.

`ColorAdjustableDevice` é uma interface (não uma classe abstracta) pois não carrega estado. Declara `setColorTemperature(int)`, `getColorTemperature()` e `getId()`. É implementada pelas classes concretas que suportam esta capacidade, independentemente da sua posição na hierarquia de herança.

2.1.3. Classes Concretas de Dispositivos

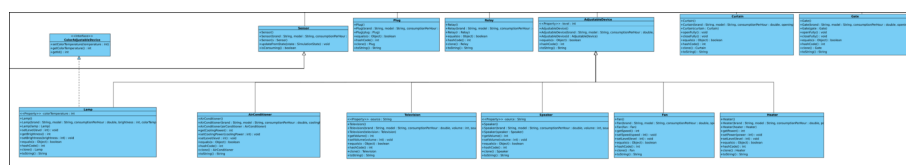


Figura 3: Diagrama de classes dos dispositivos instanciáveis

A aplicação define dez tipos de dispositivos concretos:

Relay - o dispositivo mais simples: apenas liga e desliga. Estende `SwitchableDevice`.

Plug - tomada, estende `SwitchableDevice`. Permite ligar e desligar cargas eléctricas.

Lamp - lâmpada inteligente. Estende `AdjustableDevice` (controlando o brilho de 0 a 100%) e implementa `ColorAdjustableDevice` (temperatura de cor de 2700K a 4000K). É a única classe concreta com dois graus de liberdade, o que motivou a criação da interface `ColorAdjustableDevice` em separado, evitando colocar em `AdjustableDevice` uma funcionalidade que não é universal.

Speaker - coluna de som. Estende `AdjustableDevice`, onde o nível representa o volume (0–100%).

Television - televisão inteligente. Estende `AdjustableDevice` (volume) e adiciona um campo `source` que representa a fonte de vídeo ativa (HDMI, cabo, streaming, etc.).

Fan - ventoinha com velocidade regulável. Estende `AdjustableDevice`, onde o nível representa a velocidade de 0 a 100%.

AirConditioner - ar condicionado com potência de arrefecimento regulável. Estende `AdjustableDevice`.

Heater - aquecedor com potência de aquecimento regulável. Estende `AdjustableDevice`.

Curtain - cortina inteligente. Estende `OpenableDevice`, com `openingLevel` a representar a percentagem de abertura.

Gate - portão de garagem. Estende `OpenableDevice`, funcionando de forma análoga a `Curtain`.

2.1.4. Subpacote `devices.sensors`

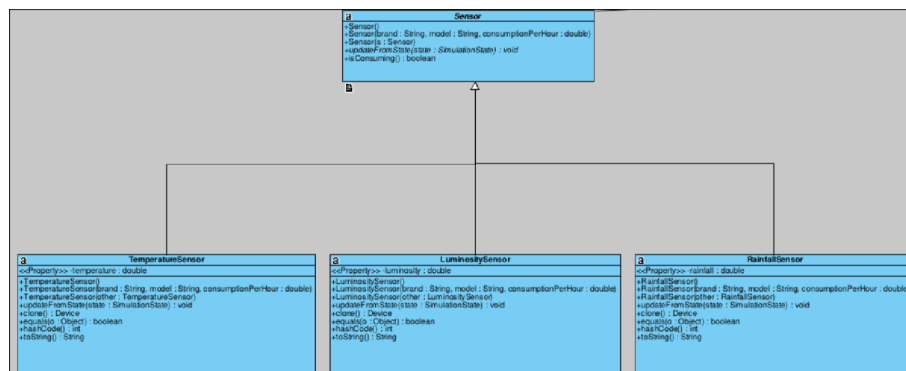


Figura 4: Diagrama de classes com a hierarquia de Sensores

O subpacote `devices.sensors` introduz um segundo tipo de dispositivo com semântica diferente dos anteriores: os sensores ambientais. Estes dispositivos não atuam sobre o ambiente - lêem o estado da simulação e disponibilizam esses valores para as condições de automação.

Sensor é a classe abstracta base de todos os sensores. Estende `SwitchableDevice` e é sempre inicializada no estado ON. O método abstracto `updateFromState(SimulationState state)` é chamado pela casa em cada tick, empurrando os dados ambientais atuais para dentro do sensor.

LuminositySensor - mede a luminosidade ambiente em lux, derivada do produto entre a luminosidade base da simulação e o multiplicador da condição meteorológica atual.

RainfallSensor - mede a intensidade de precipitação em mm/h. Devolve zero quando não chove, um valor aleatório entre 1 e 15 mm/h com chuva (**RAINING**), e entre 15 e 50 mm/h com tempestade (**STORMY**).

TemperatureSensor - lê a temperatura ambiente em graus Celsius diretamente da simulação.

2.2. Pacote simulation

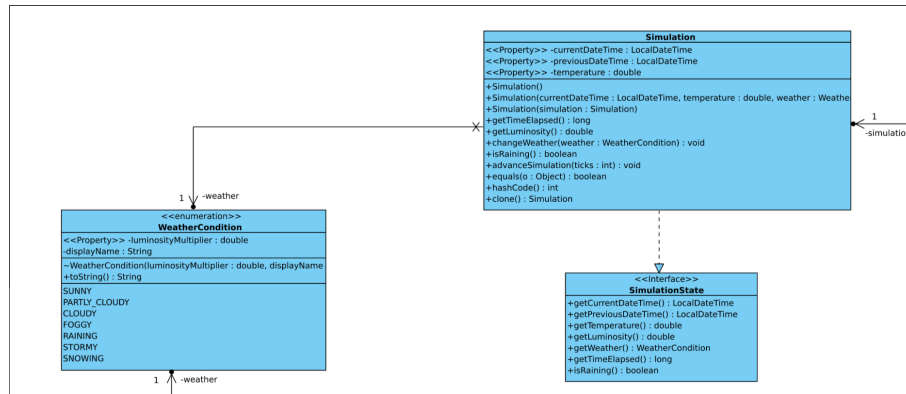


Figura 5: Diagrama de classes do pacote Simulation

O pacote **simulation** encapsula toda a lógica de simulação ambiental, que foi extraída das variáveis estáticas de **DomusControl** para uma classe de domínio própria.

Simulation é a classe central do pacote. Mantém o estado ambiental atual da simulação: **currentDateTime**, **previousDateTime**, **temperature** e **weather** ao qual está associado a **luminosity**. O método **advanceSimulation(int ticks)** avança o tempo e recalcula a luminosidade com base na condição meteorológica e na hora do dia - de noite a luminosidade é zero independentemente do tempo. O método **changeWeather(WeatherCondition weather)** permite ao utilizador alterar a condição meteorológica a qualquer momento. A classe é serializable e faz parte do estado gravado em ficheiro.

SimulationState atua como uma **read-only interface** (interface de leitura estrita) que expõe o estado atual do ambiente simulado. Em vez de se instanciar um novo objeto com um **snapshot**, a cada **tick** da simulação, o sistema clona a instância central de **Simulation** e passa-a aos sensores e rotinas tipificada estritamente como **SimulationState**. Desta forma os recetores podem consultar a hora, a temperatura, a luminosidade e o clima, mas as assinaturas de mutação (como avançar o tempo) ficam ocultas. Adicionalmente, por se tratar de um clone estrutural, mesmo que um componente tente forçar o **downcast** para a classe concreta **Simulation**, o estado real do motor de simulação permanece totalmente protegido contra modificações indevidas.

WeatherCondition é um enum com oito valores, cada um com um multiplicador de luminosidade e um nome de apresentação: **SUNNY** (1.0), **PARTLY_CLOUDY** (0.8), **CLOUDY** (0.5), **FOGGY** (0.4), **RAINING** (0.3), **STORMY** (0.15), **SNOWING** (0.6). Esta informação é usada por **Simulation** para calcular a luminosidade e por **RainfallSensor** para determinar a precipitação.

2.3. Pacote houses

2.3.1. Classe House

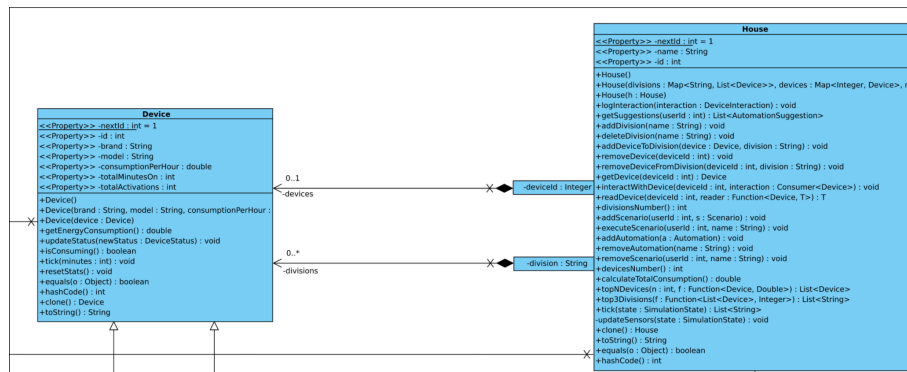


Figura 6: Diagrama de classes da class House

House é o agregador central da aplicação. Mantém dois mapas internos que trabalham em conjunto:

```
private Map<Integer, Device> devices; // dispositivos indexados por ID
private Map<String, List<Device>> divisions; // divisões com referências ao mapa anterior
```

No desenho arquitetural da classe House, optou-se deliberadamente por não criar uma classe dedicada para representar uma entidade “Divisão”. Uma divisão não possui estado próprio relevante (além do seu nome) e não tem qualquer utilidade ou existência lógica fora do contexto de uma casa. Por esta razão, as divisões são internamente representadas e geridas apenas como chaves do tipo `String`, mapeando o nome da divisão para a respetiva lista de dispositivos.

Os dispositivos nas listas de divisões são referências para os mesmos objectos presentes no mapa global - não cópias. Esta decisão foi propositada e tem duas consequências importantes. Por um lado, garante que quando um dispositivo é modificado (através de uma ação ou de forma manual), o estado atualizado é imediatamente visível em qualquer consulta sobre a divisão onde o dispositivo se encontra. Optamos por eliminar os dispositivos quando se elimina a divisão associada, no entanto, esta abordagem permitiria facilmente mudar a estratégia conseguindo eliminar divisões ficando na mesma com os dispositivos na casa (bastaria eliminar a entrada correspondente à divisão sem eliminar os dispositivos do mapa global da casa).

House contém ainda um `RoutineManager` que gere as automações, escalonamentos e cenários desta casa, e um `InteractionLogger` que regista o histórico de interações manuais para o motor de sugestões.

O acesso e a modificação do estado dos dispositivos dentro de uma casa são geridos de forma encapsulada através dos métodos `interactWithDevice` (que recebe um `Consumer`) e `readDevice` (que recebe uma `Function< Device, T>`).

A grande vantagem arquitetural desta abordagem é a passagem de comportamento por parâmetro. Em vez de expor as referências “vivas” dos dispositivos ao exterior, o que quebraria o encapsulamento e permitiria alterações indevidas de estado ou de escrever

dezenas de métodos individuais repetitivos para cada operação possível (turnOn, setLevel, getTemperature, etc.). O sistema exterior submete a operação desejada sob a forma de uma expressão lambda, e a House encarrega-se de encontrar o dispositivo, validar a sua existência e aplicar a operação internamente.

Isto elimina a duplicação de código e torna o sistema altamente extensível: se for adicionado um novo dispositivo com comportamentos totalmente novos, a classe House não precisa de sofrer qualquer alteração, bastando passar uma nova expressão lambda.

2.3.2. Classe HouseManager

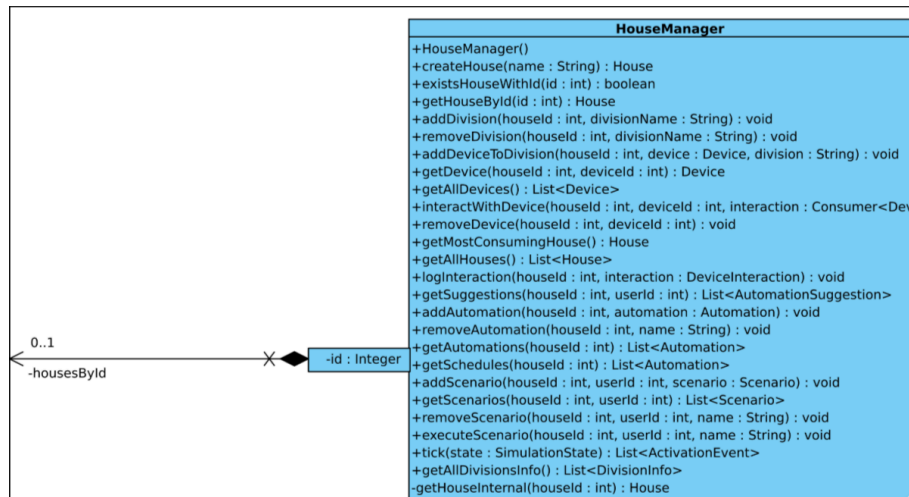


Figura 7: Diagrama de classes da class HouseManager

A classe **HouseManager** funciona como o gestor central de todas as instâncias de **House** do sistema, mantendo-as num `Map<Integer, House>`. Esta class atua como um coordenador e orquestrador global, assumindo três responsabilidades arquiteturais fundamentais:

Encapsulamento e Delegação: O gestor atua como o intermediário exclusivo entre a fachada (**DomusControl**) e as casas individuais. Foi aplicado composição, pelo que, clonamos uma *House* tanto à entrada como à saída. Implementa o método privado `getHouseInternal(int)`, que recupera a referência viva (“live”) de uma casa, distinguindo-o claramente do método público `getHouseById`, que devolve estritamente um clone. Todas as operações de modificação no sistema (como adicionar um dispositivo, registar interações ou executar cenários) passam por este gestor, que localiza a casa correta de forma segura e lhe delega a ação. Implementou-se ainda o método `interactWithDevice`, que recebe um `Consumer<Device>` e o propaga diretamente para a casa alvo.

Orquestração do Tempo de Simulação: O método `tick(SimulationState)` é responsável por propagar a passagem do tempo por todo o sistema. Itera sobre todas as casas registadas, invoca os respetivos ciclos de simulação e recolhe os resultados. Estes resultados são então encapsulados numa lista de objetos **ActivationEvent**, que mapeiam exatamente quais as automações que dispararam e em que casas, permitindo reportar esses eventos de forma agregada à interface de utilizador.

Agregação de Dados Globais: Para responder às consultas estatísticas de âmbito multi-casa (como identificar a casa de maior consumo ou calcular o top de dispositivos), foram implementados métodos como `getAllDevices()` e `getAllDivisionsInfo()` que reco-

lhem dados dispersos por todas as casas e unificam-nos em listas instanciando DTOs (*Data Transfer Objects*) como `DivisionInfo` no processo, simplificando drasticamente a lógica de apresentação e ordenação na camada superior.

2.3.3. Classe `DivisionInfo`

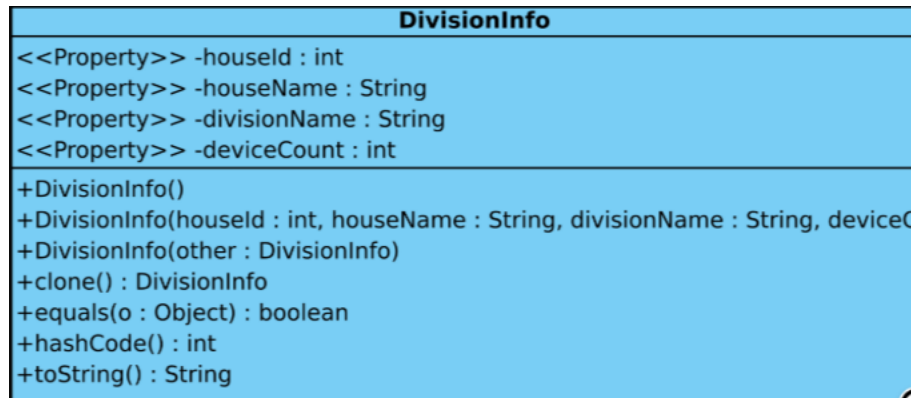


Figura 8: Diagrama de classes da class `DivisionInfo`

A classe `DivisionInfo` atua como um DTO (*Data Transfer Object*) de valor imutável, concebido para transferir dados da fachada do modelo para a camada de interface (UI) sem expor as estruturas internas de `House`.

Uma vez que as divisões são representadas internamente apenas como chaves do tipo `String` dentro de cada casa, não existe uma entidade isolada “Divisão” no modelo de domínio. Assim, quando a aplicação necessita de apresentar estatísticas transversais a todo o sistema (como o Top 3 de divisões com mais dispositivos), tornou-se estritamente necessário criar este contentor. A classe agrupa o nome da divisão com o contexto da sua casa, incluindo o `houseId` e o `houseName`. O `houseId` garante a identificação única da casa, mesmo quando existem casas com o mesmo nome, enquanto o `houseName` torna a apresentação dos resultados mais legível para o utilizador. Desta forma, evitam-se ambiguidades ou colisões de dados, permitindo distinguir, por exemplo, a “Sala” de duas casas diferentes com o mesmo nome, transportando simultaneamente a contagem de dispositivos de forma encapsulada.

2.4. Pacote routines

2.4.1. Classe Routine

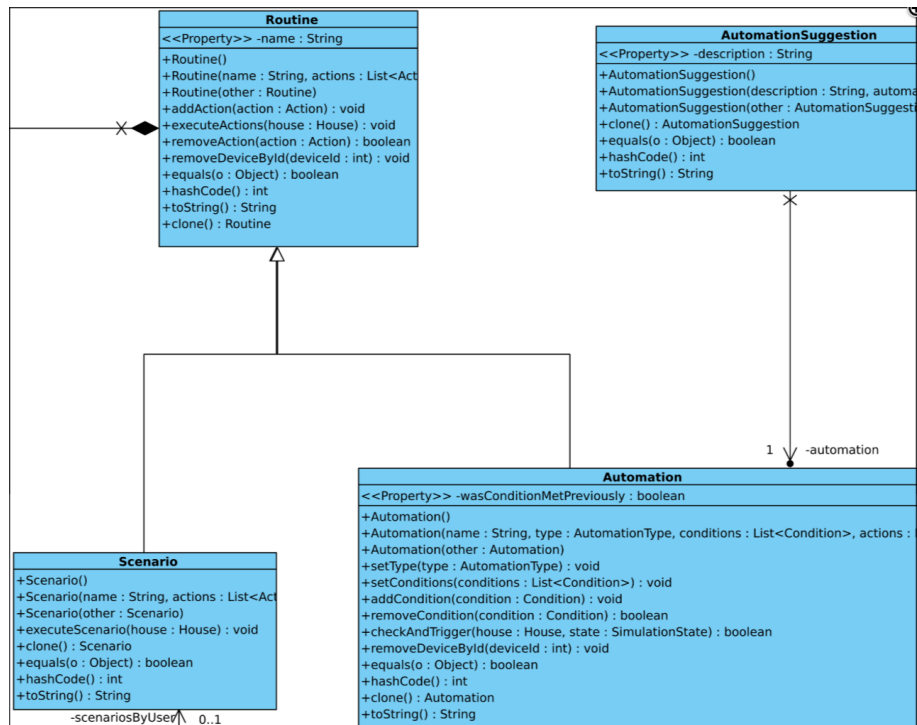


Figura 9: Diagrama de classes de Routine e a hierarquia

Routine é a classe abstracta que define o conceito base de uma rotina: um `name` e uma `List<Action>`. Define `execute()` que percorre a lista e invoca `execute()` em cada ação. Todas as subclasses herdam este comportamento de execução, o que elimina duplicação de código entre **Scenario** e **Automation**.

2.4.2. Classe Scenario

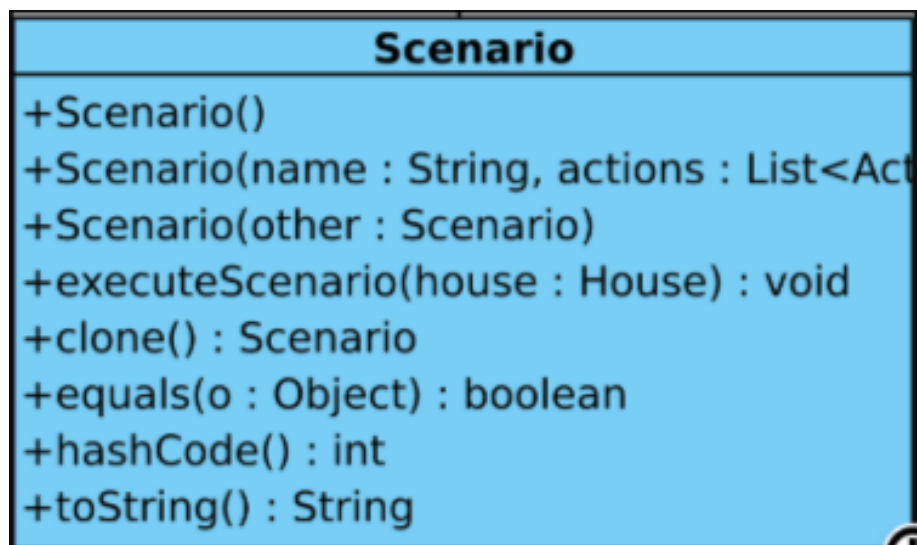


Figura 10: Diagrama de classes da classe Scenario

Scenario estende **Routine** diretamente, sem adicionar qualquer campo. Um cenário é simplesmente uma rotina nomeada que pode ser executada manualmente. O método

executeScenario() delega em execute() da superclasse. Os cenários são guardados no RoutineManager organizados por utilizador: cada utilizador tem o seu próprio Map<String, Scenario>, pois o enunciado especifica que cenários são pessoais.

2.4.3. Classe Automation

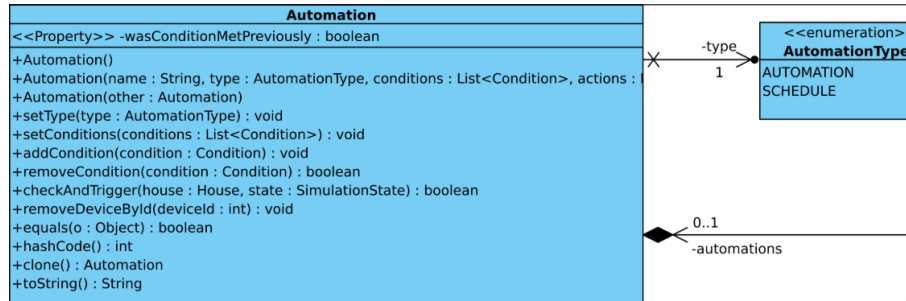


Figura 11: Diagrama de classes da classe Automation

Automation estende Routine e adiciona:

- type - AutomationType.AUTOMATION ou AutomationType.SCHEDULE. Um escalonamento é uma automação cujas condições são todas temporais.
- conditions - lista de Condition que devem ser todas verdadeiras para que as ações sejam executadas.
- wasConditionMetPreviously - booleano que regista se as condições estavam satisfeitas no tick anterior. Garante que as ações só disparam na transição de falso para verdadeiro, evitando que uma automação execute repetidamente enquanto a condição é verdade.

O construtor parameterizado lança ScheduleWithConditionDifferentFromTimeException se o tipo for SCHEDULE mas alguma condição não for uma instância de TimeBasedCondition.

2.4.4. Interface Action e Implementações

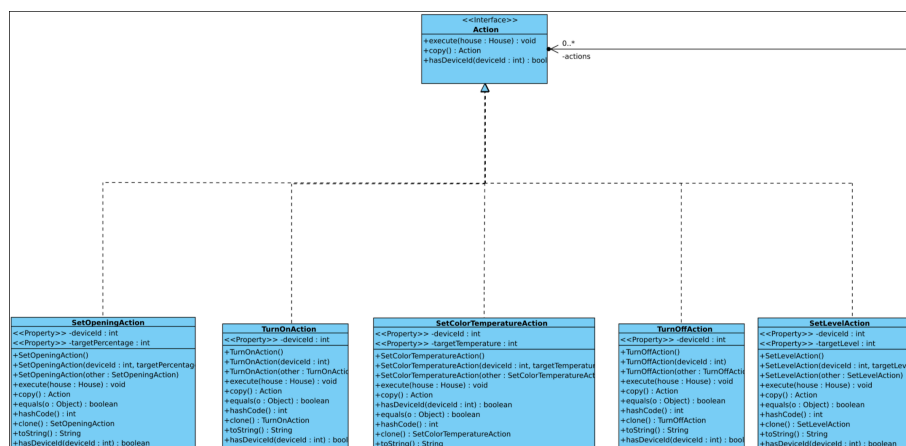


Figura 12: Diagrama de classes de Action e implementações

A interface Action declara:

- execute() - executa a ação.
- copy() - cria uma cópia da ação.

- `hasDeviceId(int deviceId)` - verifica se esta ação está associada a um dispositivo com o ID dado. Usado pelo `RoutineManager` para limpar referências quando um dispositivo é removido.

As cinco implementações concretas da interface `Action` são `TurnOnAction`, `TurnOffAction`, `SetLevelAction`, `SetOpeningAction` e `SetColorTemperatureAction`. Cada ação guarda estritamente o ID do dispositivo alvo e os parâmetros específicos da operação (como a porcentagem de abertura ou o nível de brilho), não mantendo qualquer referência direta para o objeto do dispositivo em si.

A decisão de guardar apenas o identificador e injetar a referência da casa (`House`) no momento da execução (através do `RoutineManager`) permite desacopla-las das ações do ciclo de vida dos dispositivos: evita-se o risco de manter referências “fantasma” na memória caso um dispositivo seja removido do sistema. Simplifica drasticamente a clonagem e a serialização das rotinas, tornando-as em objetos de dados leves e sem estado de domínio. Quando uma automação dispara, a ação consulta a casa em tempo de execução para atuar sobre o dispositivo, garantindo que opera sempre o objeto original (o mapa central de dispositivos vivos) e com o estado mais atualizado do sistema.

2.4.5. Interface `Condition` e Implementações

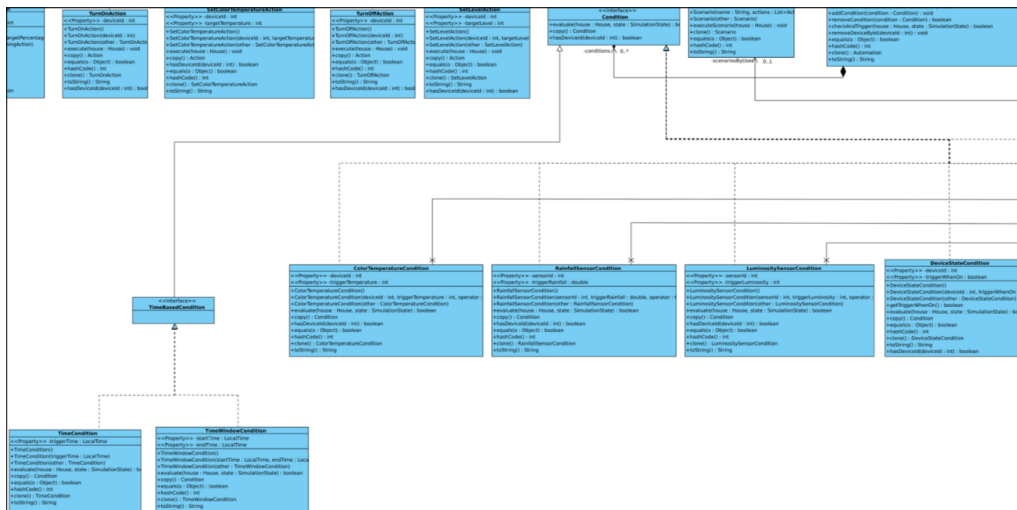


Figura 13: Diagrama de classes de Condition e suas implementações (parte 1)

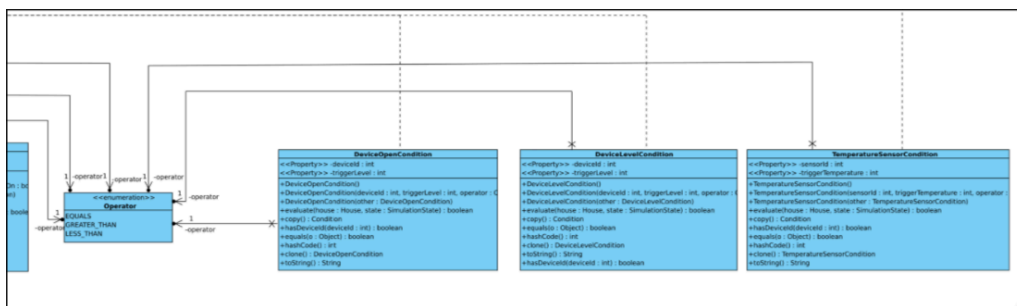


Figura 14: Diagrama de classes de Condition e suas implementações (parte 2)

A interface `Condition` declara:

- `evaluate()` - avalia a condição e devolve um booleano.
- `copy()` - cria uma cópia da condição.

- `hasDeviceId(int deviceId)` - análogo ao das ações.

As implementações concretas cobrem todos os tipos de condição suportados:

DeviceStateCondition - recebe um `SwitchableDevice` e um booleano `triggerWhenOn`. Avalia para verdadeiro se `device.isOn() == triggerWhenOn`.

DeviceLevelCondition - recebe um `AdjustableDevice`, um `triggerLevel` inteiro e um `Operator` (`EQUALS`, `GREATER_THAN`, `LESS_THAN`). Avalia comparando `device.getLevel()` com o valor de referência usando o operador configurado.

DeviceOpenCondition - análoga à anterior mas para `OpenableDevice`, comparando `device.getOpeningLevel()`.

ColorTemperatureCondition - análoga mas para `ColorAdjustableDevice`, comparando `device.getColorTemperature()`.

TimeCondition - verifica se o instante configurado se encontra no intervalo `[previousDateTime, currentDateTime]`. Isto simula corretamente o disparo pontual: se o utilizador avança 30 minutos de uma vez e o escalonamento está configurado para as 14:15, ele dispara mesmo que o tick não tenha parado exactamente às 14:15.

TimeWindowCondition - verifica se a hora atual se encontra dentro de uma janela `[start, end]`. Útil para regras do tipo “a luz deve estar ligada entre as 18:00 e as 23:00”.

Estas duas últimas implementam a interface `TimeBasedCondition`, que serve como marcador para identificar condições temporais. Esta interface é usada pelo construtor de `Automation` para validar que um escalonamento apenas contém condições deste tipo - em vez de verificar cada classe concreta individualmente, basta verificar se a condição é uma instância de `TimeBasedCondition`. Isto torna a verificação de ver se uma condição é baseado no tempo independente do número de tipos de condições temporais existentes: adicionar uma nova condição temporal resume-se a implementar a interface, sem ter que acrescentar nada às condições de verificação.

LuminositySensorCondition - recebe um `LuminositySensor` e compara a leitura de luminosidade (em lux) com um valor de referência usando um `Operator`. Permite criar automações do tipo “se a luminosidade medida for inferior a 200 lux, ligar as luzes da sala”.

RainfallSensorCondition - recebe um `RainfallSensor` e compara a intensidade de precipitação (em mm/h) com um valor de referência. Permite criar automações do tipo “se estiver a chover, fechar as cortinas”.

TemperatureSensorCondition - recebe um `TemperatureSensor` e compara a temperatura ambiente (em °C) com um valor de referência. Permite criar automações do tipo “se a temperatura medida for inferior a 15°C, ligar o aquecedor”.

2.4.6. Classe **RoutineManager**

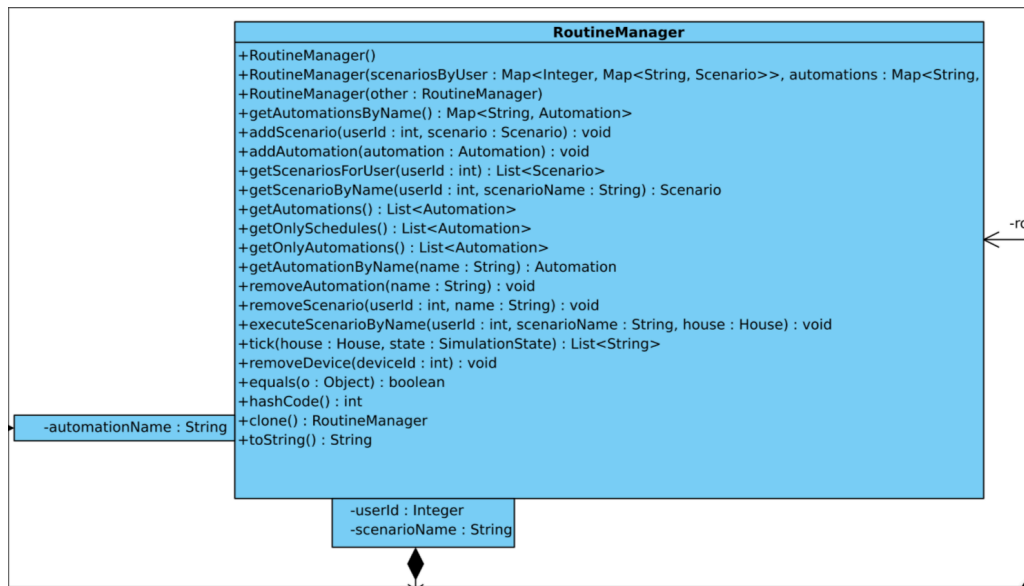


Figura 15: Diagrama de classes da classe **RoutineManager**

RoutineManager é o gestor central de todas as rotinas de uma casa. Mantém:

- `Map<String, Automation> automations` - automações e escalonamentos, indexados pelo nome.
- `Map<Integer, Map<String, Scenario>> scenariosByUser` - cenários organizados por ID de utilizador e depois por nome.

O método `tick()` percorre todas as automações e invoca `checkAndTrigger()` em cada uma. Este método avalia as condições e, se transitaram de falso para verdadeiro, executa as ações e devolve o nome da automação disparada. `RoutineManager.tick()` devolve `List` com os nomes das automações disparadas, em vez de `ActivationEvent`, porque não tem acesso ao contexto da casa em que está inserido. A conversão para `ActivationEvent` ocorre em `HouseManager.tick()`, que detém esse contexto e é a camada responsável por agregar a informação de casa e rotina num único objecto. Esta separação evita acoplar `RoutineManager` a conceitos de outras camadas, respeitando o princípio da responsabilidade única. O `DomusControl` recolhe estes `ActivationEvent` entregando-os à UI que mostra quais rotinas foram executadas em cada avanço de tempo.

O método `removeDevice(int deviceId)` percorre todas as automações e cenários e remove quaisquer ações ou condições que referenciem o dispositivo. Isto garante que remover um dispositivo não deixa referências inválidas no sistema.

2.5. Pacote suggestions

2.5.1. InteractionType

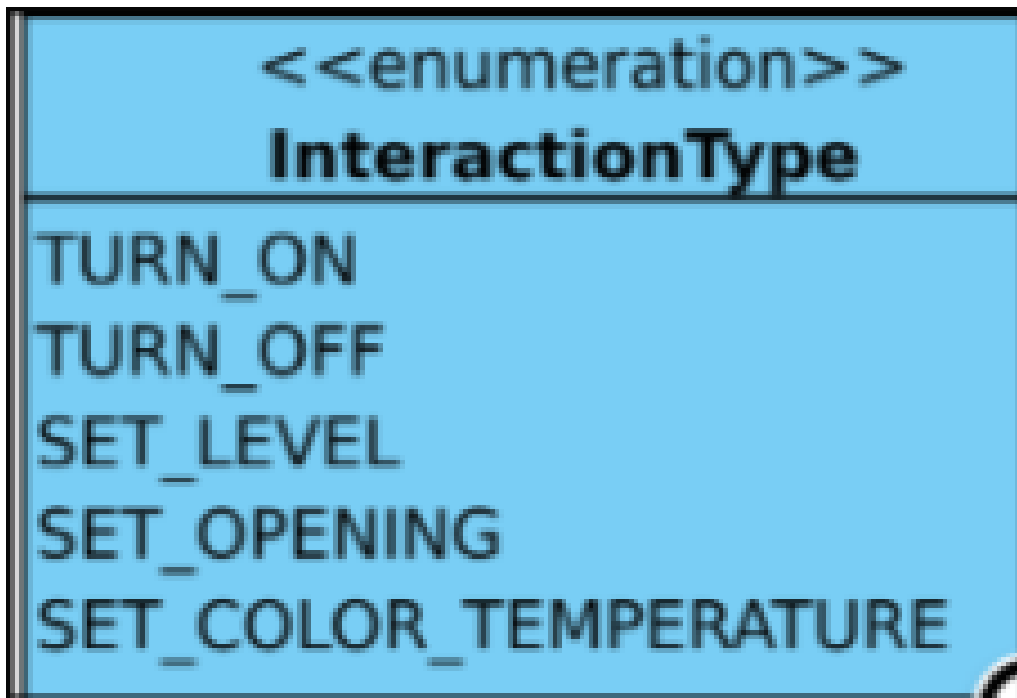


Figura 16: Diagrama de classes da classe Interaction Type

Enum com os cinco tipos de interação possíveis: **TURN_ON**, **TURN_OFF**, **SET_LEVEL**, **SET_OPENING**, **SET_COLOR_TEMPERATURE**.

2.5.2. Classe DeviceInteraction

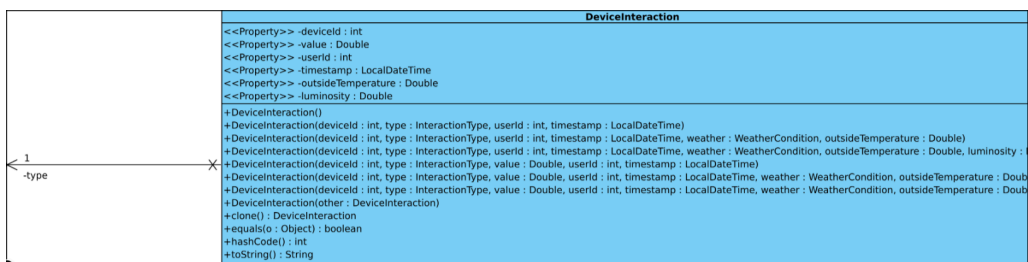


Figura 17: Diagrama de classes da classe DeviceInteraction

A classe **DeviceInteraction** encapsula o registo de uma operação manual realizada sobre um dispositivo. Mais do que guardar apenas a ação (ID do dispositivo, tipo de interação e valor) e o seu autor (ID do utilizador), foi adicionado também o contexto ambiental.

No momento exato em que a interação ocorre, a classe regista um **snapshot** do estado da simulação. Isto inclui o instante exato (**timestamp**), o estado meteorológico (**weather**), a temperatura exterior (**outsideTemperature**) e a luminosidade (**luminosity**). É a injeção destes dados climáticos no momento da ação humana que fornece ao motor de sugestões a informação detalhada necessária para conseguir correlacionar hábitos de utilização com fatores ambientais.

2.5.3. InteractionLogger

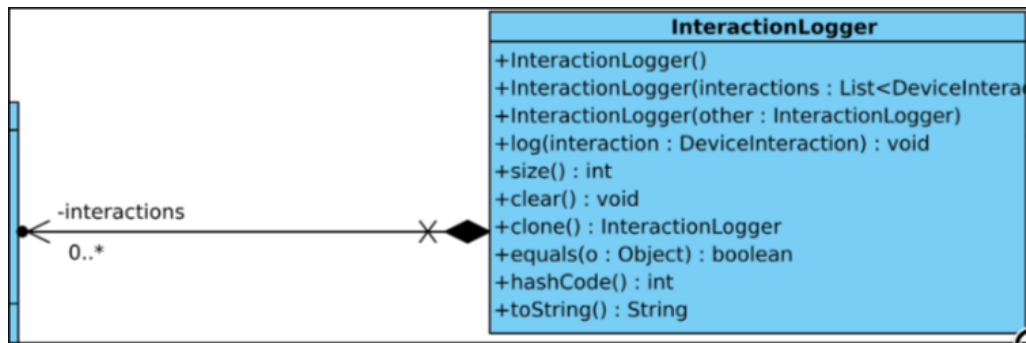


Figura 18: Diagrama de classes da classe InteractionLogger

Mantém uma `List<DeviceInteraction>` em ordem cronológica de inserção. O método `log(DeviceInteraction)` acrescenta uma entrada. O método `getInteractions()` devolve uma vista não modificável da lista, preservando a ordem cronológica necessária para a detecção de sequências. Cada instância de `House` possui o seu próprio `InteractionLogger`, o que limita naturalmente a análise ao histórico daquela casa.

2.5.4. Classe SuggestionEngine

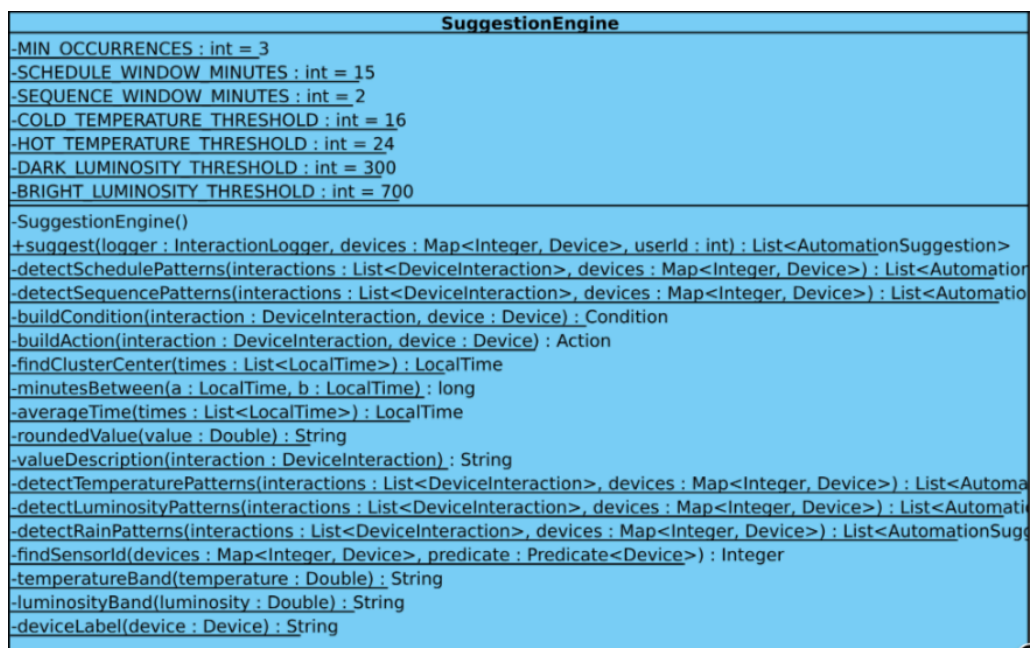


Figura 19: Diagrama de classes da classe Suggestion Engine

Classe utilitária com construtor privado, responsável por analisar o histórico e sugerir automações inteligentes. O método estático `suggest(InteractionLogger, Map<Integer, Device>, int userId)` recebe o **logger**, o mapa de dispositivos da casa e o ID do utilizador, extraindo apenas as interações relevantes a esse utilizador para devolver uma `List<AutomationSuggestion>`. O motor deteta agora cinco categorias de padrões:

A deteção de padrões temporais agrupa interações por (`deviceId`, `type`, `roundedValue`) e verifica se algum grupo tem três ou mais entradas cujas horas do dia formam um **cluster** com desvio máximo de quinze minutos. O centro do **cluster** é calculado como média

circular dos instantes, e um `TimeCondition` é construído com essa hora, resultando num escalonamento (**Schedule**) sugerido.

A detecção de padrões de sequência procura pares de interações (A, B) onde B ocorre num dispositivo diferente no máximo dois minutos depois de A, com pelo menos três ocorrências. Para cada par suficientemente frequente, constrói uma `Automation` com a condição correspondente ao estado de A e a ação correspondente ao estado de B. Qualquer tipo de interação pode ser o gatilho, não limitando o sistema a dispositivos on/off.

Foi adicionado também a detecção de padrões ambientais e climáticos de forma a expandir a inteligência do sistema, correlacionando interações com as condições ambiente:

Temperatura: Regista se uma ação ocorre consistentemente em ambientes frios (abaixo de 16°C) ou quentes (acima de 24°C).

Luminosidade: Deteta padrões quando o ambiente está escuro (abaixo de 300 lx) ou muito claro (acima de 700 lx).

Chuva: Verifica se uma ação é repetida quando o estado meteorológico indica precipitação ou tempestade.

Para que estas sugestões climáticas sejam válidas e geradas, é procurado dinamicamente nos dispositivos da casa a existência do sensor adequado (`TemperatureSensor`, `LuminositySensor` ou `RainfallSensor`) através do método `findSensorId`, associando a condição de disparo ao ID do sensor físico real. Caso não exista o sensor não são criadas as automações que necessitavam da sua existência.

Em todos os casos, as ações e condições finais são construídas utilizando os métodos internos `buildAction` e `buildCondition`, que convertem o registo de `DeviceInteraction` nas instâncias concretas corretas. Estes métodos garantem segurança de tipos, verificando se os dispositivos alvo implementam as interfaces de capacidade exigidas (ex: `AdjustableDevice`, `OpenableDevice`) antes de gerar a sugestão.

2.5.5. AutomationSuggestion

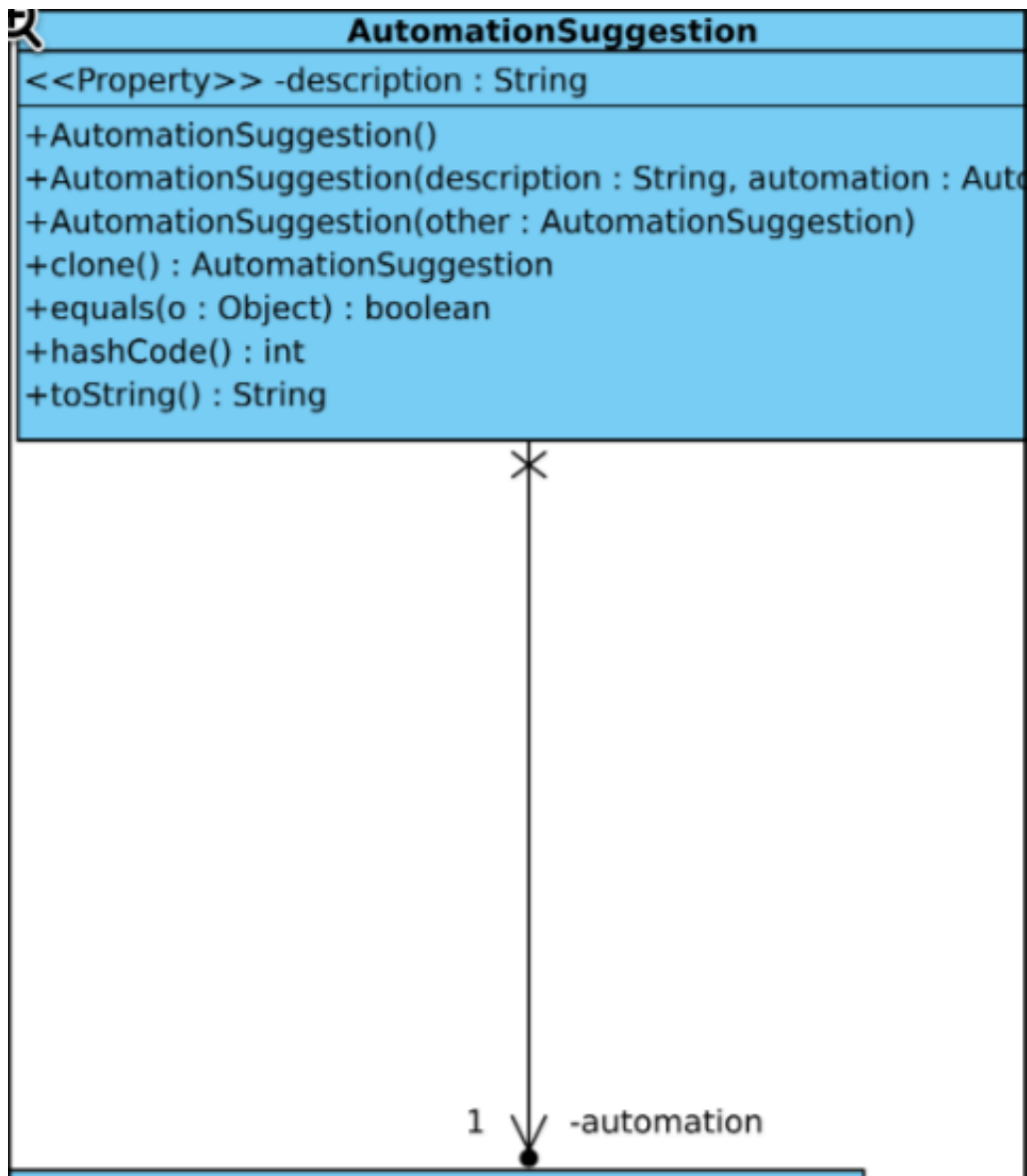


Figura 20: Diagrama de classes da classe AutomationSuggestion

Contém uma descrição legível por humanos da sugestão e a **Automation** já instanciada e pronta a ser adicionada ao **RoutineManager** se o utilizador a aceitar.

2.6. Pacote user

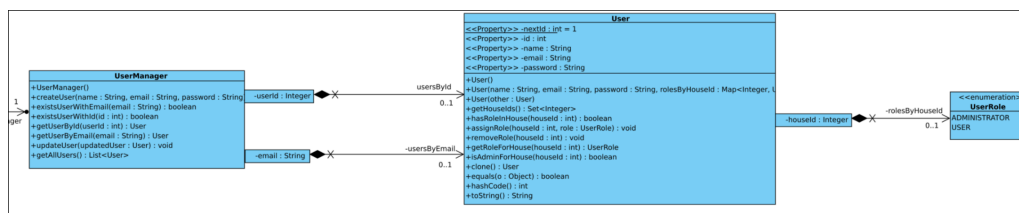


Figura 21: Diagrama de classes da Package Users

2.6.1. Enum `UserRole`

Define dois papéis: `ADMIN` e `USER`. Guardado por `User` num `Map<Integer, UserRole>` que associa IDs da casa a papéis. O `DomusControl` consulta este mapa para condicionar opções de menu - apenas utilizadores com papel `ADMINISTRADOR` podem gerir divisões, dispositivos e outros utilizadores da casa.

2.6.2. Classe `User`

Armazena email, nome e password. O email é o identificador de login e serve como chave no `UserManager`. A sua responsabilidade arquitetural mais importante é a gestão do controlo de acessos.

Para suportar o requisito de um utilizador poder aceder a múltiplas casas com privilégios distintos, a classe tem um `Map<Integer, UserRole>` que associa o ID de cada casa ao respetivo papel do utilizador (`ADMINISTRATOR` ou `USER`). Este design descentraliza a lógica de autorização: a entidade sabe responder de forma autónoma ao seu próprio nível de privilégio num dado contexto através de métodos como `isAdminForHouse(houseId)`. Desta forma a Interface (UI), que apenas necessita de interrogar o utilizador logado para decidir se deve, ou não, apresentar opções de gestão exclusivas a administradores para a casa atualmente selecionada.

2.6.3. Classe `UserManager`

Mantém um `Map<String, User>` indexado por email e outro mapa indexado por id. Fornece métodos de criação, validação de login, recuperação por email ou ID, e atualização de dados. Foi aplicado composição, ou seja, é feito um clone tanto à entrada como à saída de um `User`. O método `validateLogin` lança `UserNotFoundException` se o email não existir e `LoginInvalidPasswordException` se a password estiver errada, permitindo à UI distinguir os dois casos e apresentar mensagens apropriadas.

2.7. Pacote `exceptions`

Foram definidas vinte classes de exceção específicas, de forma a tornar mais claro e expressivos os erros, sendo que todas estendem `Exception`:

- `DeviceNotFoundException`, `DivisionNotFoundException` - utilizadas quando dispositivo e divisão não existem e foram procuradas.
- `HouseNotFoundException`, `HouseAlreadyExistsException` - gestão de casas.
- `UserNotFoundException`, `UserAlreadyExistsException` - gestão de utilizadores.
- `LoginInvalidPasswordException` - autenticação falhada por password errada.
- `UserNotLoggedInException` - operação tentada sem sessão ativa.
- `LastAdminException` - tentativa de remover o último administrador de uma casa.
- `NameAlreadyExistsException` - nome de automação, cenário ou escalonamento duplicado.
- `InvalidMinutesException` - tentativa de avançar o tempo de simulação com um valor inválido (ex: minutos negativos).
- `ScheduleWithConditionDifferentFromTimeException` - escalonamento construído com condição não temporal.

- `AutomationDoesntExistException`, `ScenarioDoesntExistException`, `ScheduleDoesntExistException` - remoção ou execução de rotinas inexistentes.
- `UserDoesntHaveScenarios` - utilizador sem cenários definidos.
- `DeviceIsNotInstanceOfException` - quatro exceções específicas (`AdjustableDevice`, `ColorAdjustableDevice`, `OpenableDevice` e `SwitchableDevice`) lançadas quando se tenta realizar uma operação inválida num dispositivo que não implementa a interface/capacidade necessária.

A proliferação controlada de exceções específicas, em vez de exceções genéricas (`Exception`), permite à camada de interface (UI) capturar os diferentes erros individualmente e apresentar mensagens de erro precisas ao utilizador, garantindo ao mesmo tempo que o compilador verifica se todos os casos de falha do negócio são devidamente tratados. Os erros são propagados até chegarem à UI que informa o utilizador.

2.8. Camada de Interface

A interface com o utilizador é implementada em modo texto através de quatro classes UI e uma classe `Menu`.

2.9. Classe Menu

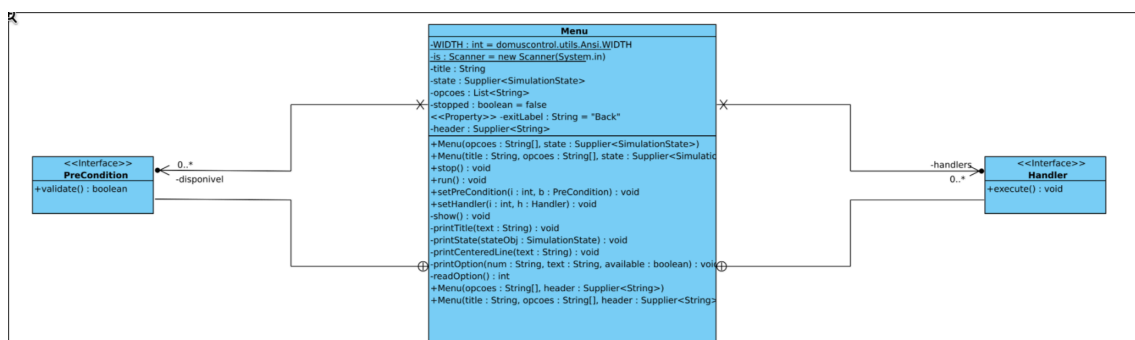


Figura 22: Diagrama de classes de Menu

A classe `Menu` é uma adaptação e extensão flexível do código fornecido pela equipa docente. Recebe um array de **strings** com os nomes das opções e um título, suportando a atribuição dinâmica de executáveis (`setHandler`) e restrições de disponibilidade (`setPreCondition`) por índice.

O grande diferencial arquitetural desta implementação é a injeção de um `Supplier<String>` para a renderização do cabeçalho. Isto permite que, em cada iteração do ciclo principal, o menu recupere e apresente dinamicamente informação atualizada (como o tempo de simulação, meteorologia e temperatura). Crucialmente, ao receber um fornecedor de **strings** genérico em vez do modelo de domínio diretamente (`SimulationState`), a classe `Menu` mantém-se pura, genérica e totalmente desacoplada da lógica de negócio.

2.9.1. Classe `DomusControlUI`

Ponto de entrada da aplicação. Inicializa o modelo e apresenta o menu inicial com opções de login, registo, estatísticas globais e carregamento de estado. Após autenticação bem-sucedida, delega no `UserUI`.

2.9.2. Classe UserUI

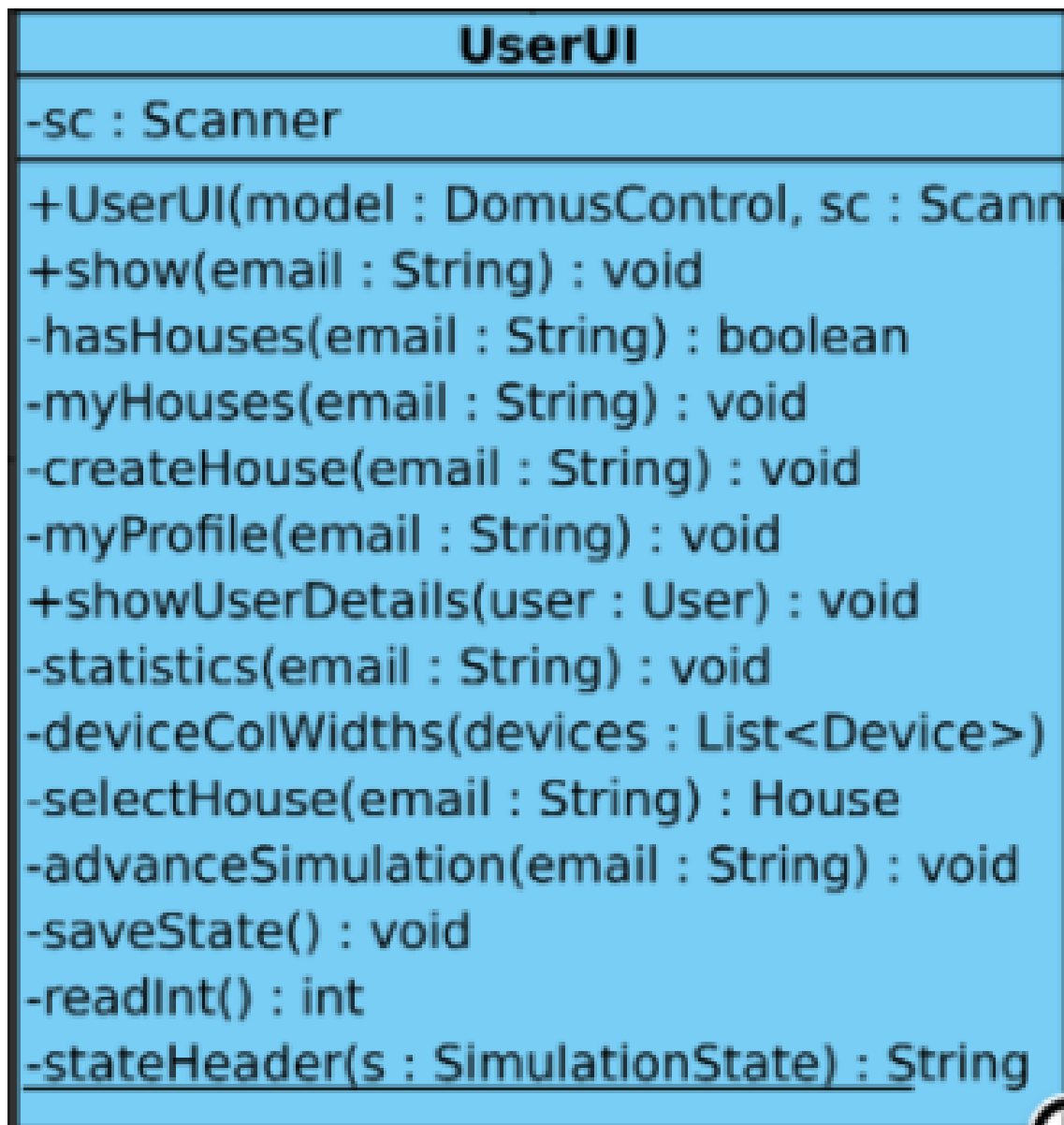


Figura 23: Diagrama de classes da classe UserUI

Gere o menu principal do utilizador autenticado: listar as suas casas, criar uma nova casa, ver e editar o seu perfil, consultar estatísticas do sistema e da sua residência, avançar o tempo de simulação e gravar o estado em ficheiro. Ao seleccionar uma casa, delega no HouseUI.

2.9.3. Classe HouseUI



Figura 24: Diagrama da classes de HouseUI

O menu principal da casa. Apresenta opções consoante o papel do utilizador (recorrendo às pré-condições do menu baseadas em `isAdmin` e `houseHasDevices`). Implementa a gestão de divisões, dispositivos, utilizadores da casa, visualização de sugestões do motor inteligente e a operação manual e individual de cada dispositivo. Para a gestão de rotinas, delega no `ActionsUI`.

2.9.4. Classe ActionsUI

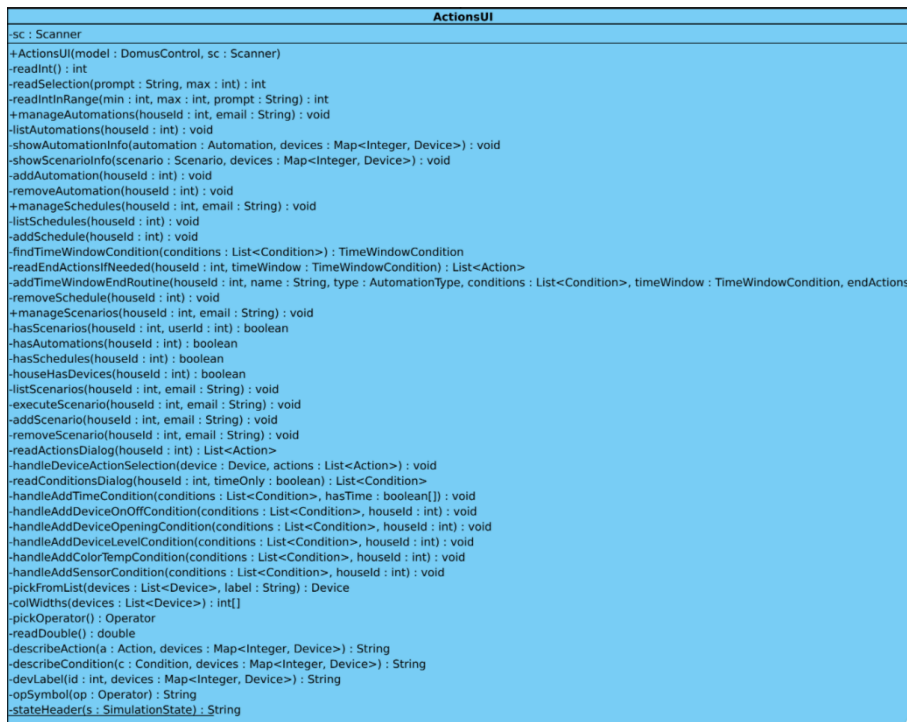


Figura 25: Diagrama de classes da classe ActionsUI

Classe extraída de HouseUI para gerir especificamente as automações, escalonamentos e cenários, aplicando o Princípio da Responsabilidade Única para evitar criar uma classe UI massiva. Apresenta menus dedicados para listar, criar e remover cada tipo de rotina, incluindo a seleção interativa de condições (estado de dispositivo, valor de sensor, temporais) e ações (ligar, definir nível, etc.) para construir automações complexas passo a passo.

3. Implementação

3.1. Gestão do Tempo e Ambiente de Simulação

O tempo de simulação, anteriormente mantido em variáveis estáticas de `DomusControl`, foi encapsulado na classe `Simulation`. Esta contém `currentDateTime`, `previousDateTime`, `temperature`, `weather` e `luminosity`. O método `advanceSimulation(int ticks)` avança o tempo e recalcula a luminosidade base combinando a hora do dia com o multiplicador da condição meteorológica.

Quando o utilizador avança o tempo através da UI, `DomusControl` invoca `simulation.advanceSimulation(minutes)`, clona a `Simulation` e é feito o *cast* para `SimulationState` e propaga-o a todas as casas. Cada casa passa-o todos os seus sensores via `updateFromState`, e depois invoca `tick` no `RoutineManager` para avaliar as automações.

O método devolve os nomes das automações que dispararam durante o tick.

3.2. Consistência de Referências em House

Uma das decisões de implementação mais importantes foi a forma como `House` mantém a consistência entre o mapa global de dispositivos e as listas de divisões. Quando um dispositivo é adicionado a uma divisão com `addDeviceToDivision`, o código garante que a referência guardada na lista da divisão é a mesma que está no mapa global:

```
public void addDeviceToDivision(Device device, String division)
    throws DivisionNotFoundException {
    Device canonical = this.devices.computeIfAbsent(
        device.getId(), k -> device.clone());
    List<Device> list = this.divisions.get(division);
    if (list.stream().noneMatch(d -> d.getId() == device.getId())) {
        list.add(canonical); // referência partilhada, não cópia
    }
}
```

Desta forma qualquer ação que actue sobre o dispositivo Atualiza o mesmo objecto que está visível nas listagens de divisões e no cálculo de consumo.

3.3. Serialização e Referências Partilhadas

A persistência do estado é feita por serialização binária Java. Toda a hierarquia de classes do modelo implementa `Serializable`. O estado completo é gravado num único `ObjectOutputStream`:

```
public void saveState(String fileName) throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(
        new FileOutputStream(fileName))) {
        out.writeObject(this);
    }
}
```

O mecanismo de handle table do Java garante que instâncias partilhadas são escritas uma única vez. Ao deserializar, as referências são restauradas com fidelidade - a `TurnOnAction` e o mapa `devices` da casa apontam novamente para o mesmo objecto. Aquando do carregamento, os contadores estáticos de `Device` e `House` são repostos para valores superiores aos IDs existentes no estado carregado, evitando colisões com entidades criadas posteriormente.

3.4. Registo de Interações para Sugestões

O registo das interações manuais é feita através de duas operações distintas:

```
public void interactWithDevice(int deviceId, Consumer<Device> interaction)
throws DeviceNotFoundException {
    Device realDevice = this.devices.get(deviceId);
    if (realDevice == null) throw new DeviceNotFoundException("'" + deviceId);
    interaction.accept(realDevice);
}

public void logInteraction(DeviceInteraction interaction) {
    this.interactionLogger.log(interaction);
}
```

Neste fluxo, a responsabilidade de orquestração pertence à camada superior (a fachada `DomusControl`). Quando o utilizador opera manualmente um dispositivo através da UI, a fachada invoca o `interactWithDevice` injetando o comportamento (a operação específica a realizar) através de uma expressão lambda (`Consumer`). Logo de seguida, a fachada constrói o objeto `DeviceInteraction`, capturando o instante exato da simulação, e passa-o ao método `logInteraction`.

3.5. Encapsulamento e Cópias Defensivas

Uma convenção seguida em todas as classes do modelo é que os getters de colecções devolvem sempre uma cópia (ou uma vista não modificável), e os setters que recebem colecções fazem cópias defensivas antes de as armazenar. A regra complementar é que nenhuma classe acede diretamente a campos de outra instância que foi implementadas através de variáveis `private`.

3.6. Estatísticas

As consultas estatísticas são implementadas com streams Java na classe `DomusControl`, usando métodos auxiliares genéricos que aceitam critérios como parâmetros funcionais:

```
public List<Device> getTopDevicesByCriterion(int n, Function<Device, Double>
criterion) {
    return this.houseManager.getAllDevices().stream()
        .sorted(Comparator.comparingDouble(criterion::apply).reversed())
        .limit(n)
        .collect(Collectors.toList());
}
```

Este método único serve tanto para ordenar por `totalMinutesOn` como por `totalactivations`, bastando passar a referência de método adequada.

4. Arquitetura MVC Delegate

4.1. Conceito

4.1.1. Conceito

O padrão clássico Model-View-Controller (MVC) organiza uma aplicação em três camadas com responsabilidades estritamente distintas (Modelo, Vista e Controlador). No entanto, no contexto deste projeto, o ponto de partida para a construção da interface de consola foi a classe `Menu` fornecida pela equipa docente. Uma vez que essa classe junta a apresentação das opções no ecrã (*output*) e a leitura imediata das escolhas do utilizador (*input*), tornou-se lógico e estruturalmente mais coerente adaptar a nossa arquitetura para acomodar este comportamento.

Por este motivo, adota-se uma variação conhecida como **MVC Delegate**. Neste padrão as vistas e o controlador unem-se:

O **Modelo** encapsula todo o estado, os dados e as regras de negócio da aplicação. Continua a ser completamente independente e “cego” em relação à forma como a informação é mostrada ou como o utilizador interage com o sistema. O **View-Controller** atua simultaneamente como apresentador e intermediário. Tirando partido da mecânica da classe `Menu`, este delegado é responsável por desenhar a interface no ecrã (Vista), capturar e validar as entradas do utilizador (Controlador) e, por fim, delegar a execução da lógica de negócio na Fachada do Modelo.

4.2. Implementação no DomusControl

No DomusControl, esta arquitetura reflete-se numa separação clara em duas grandes frentes:

O **Modelo** é constituído por `DomusControl` (a fachada), `HouseManager`, `UserManager`, `Simulation`, todas as classes de domínio (`House`, `Device`, `Routine`, `Sensor`, etc.) e o motor de sugestões. O modelo não conhece qualquer classe de interface - não tem referências a `Scanner`, não faz `System.out.println`, não sabe se está a correr num terminal ou numa aplicação gráfica.

A **Vista/Controlador** é implementada pelas classes `DomusControlUI`, `UserUI`, `HouseUI` e `ActionsUI`, com recurso às classes `Menu` fornecidas no âmbito da Unidade Curricular. Estas classes funcionam como a infraestrutura de interface, abstraindo a lógica de apresentação de opções, leitura de escolhas e execução de handlers. Neste projeto, dado o contexto de aplicação em modo texto, as responsabilidades de vista e controlador estão fundidas nas mesmas classes UI. Cada classe UI recolhe dados do utilizador, valida o formato das entradas, e delega ao modelo. Apresenta os resultados e as mensagens de erro diretamente no terminal.

Camada	Classes	Responsabilidade
--------	---------	------------------

Modelo	DomusControl, HouseManager, UserManager, Simulation, House, Device, Routine, ...	Estado, regras de negócio, persistência
Vista/Controlador	DomusControlUI, UserUI, HouseUI, ActionsUI, Menu	Entrada do utilizador, apresentação, menus
Utilitários	Ansi	Formatação de terminal

4.2.1. Fluxo de uma Operação Típica

Para ilustrar o fluxo arquitetural, considere-se a operação de ligar/desligar (toggle) um dispositivo através da interface:

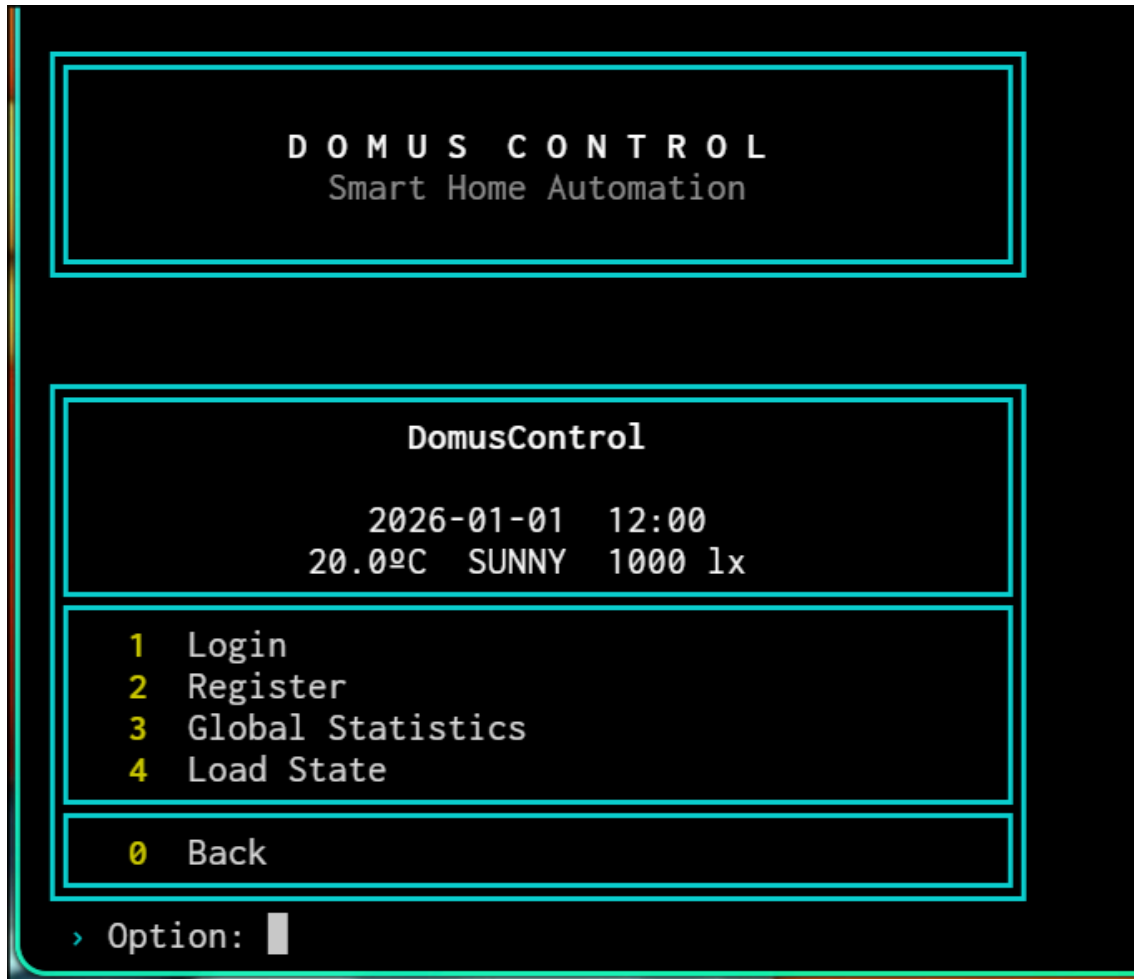
1. HouseUI apresenta a lista de dispositivos disponíveis e pede ao utilizador que selecione um.
2. O utilizador introduz o ID do dispositivo.
3. HouseUI invoca `model.toggleDevice(houseId, deviceId, userId)`.
4. DomusControl (a Fachada) orquestra a operação: delega no HouseManager a chamada a `interactWithDevice`, injetando a operação pretendida através de uma expressão **lambda** (ex: `d -> ((SwitchableDevice) d).toggle()`).
5. O HouseManager localiza a referência viva da House e passa-lhe o **consumer**. A House executa a operação em segurança, alterando o estado real do dispositivo.
6. Após a execução bem-sucedida, DomusControl constrói um objeto `DeviceInteraction` (capturando o instante atual da simulação) e invoca `logInteraction` no HouseManager, que por sua vez o guarda no `InteractionLogger` da casa.
7. Se ocorrer uma exceção em qualquer ponto (ex: `DeviceNotFoundException` ou `DeviceIsNotInstanceOfSwitchableDeviceException`), a execução do **consumer** falha antes do registo e a exceção propaga-se de volta.
8. HouseUI captura a exceção e apresenta uma mensagem de erro precisa ao utilizador; em caso de sucesso, apresenta a confirmação da operação (“Device toggled.”).

5. Utilização da Aplicação

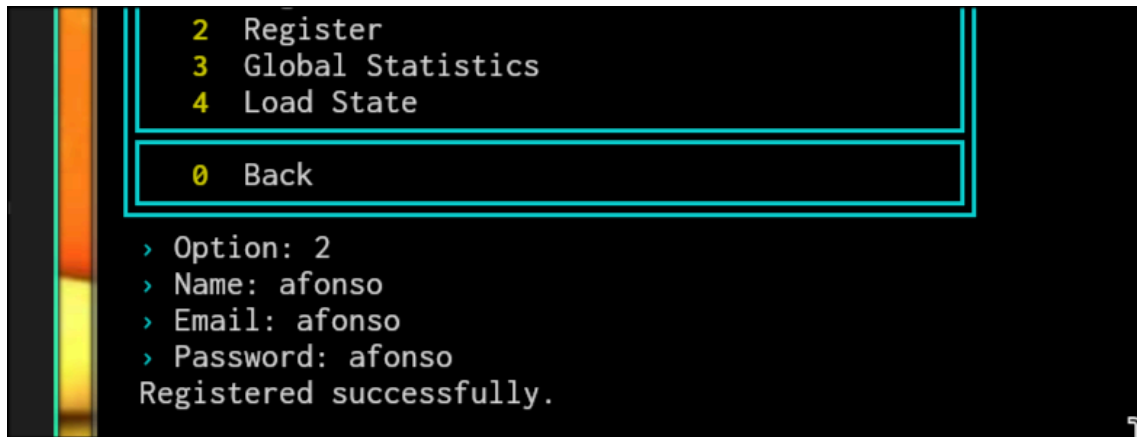
5.1. Compilação e Execução

```
./gradlew build  
./gradlew run --console=plain
```

5.2. Ecrã de Boas-Vindas

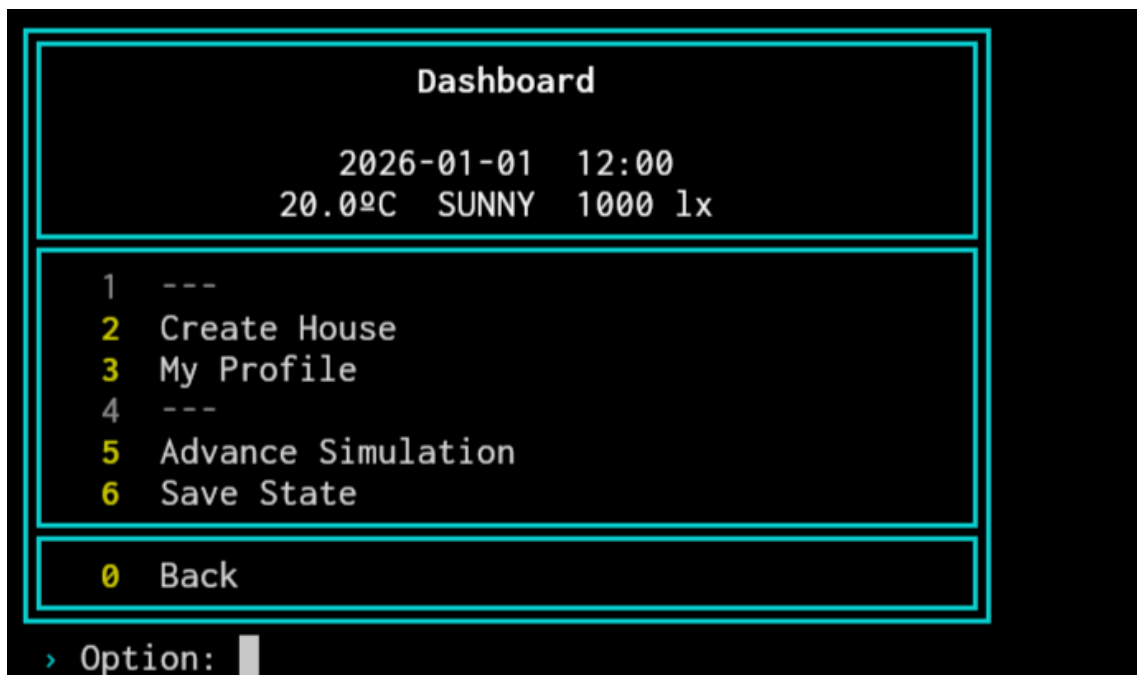


5.3. Registo e Login

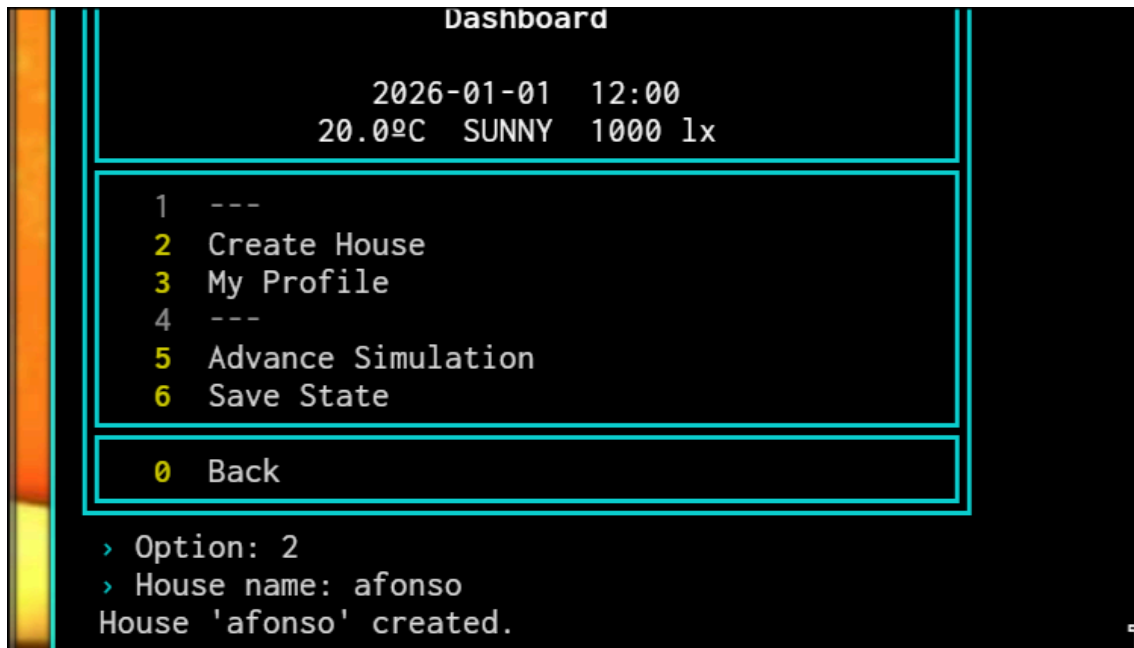


O login valida o email e a password. Se o email não existir é apresentada a mensagem “*User not found*”; se a password estiver errada, “*Invalid password*”.

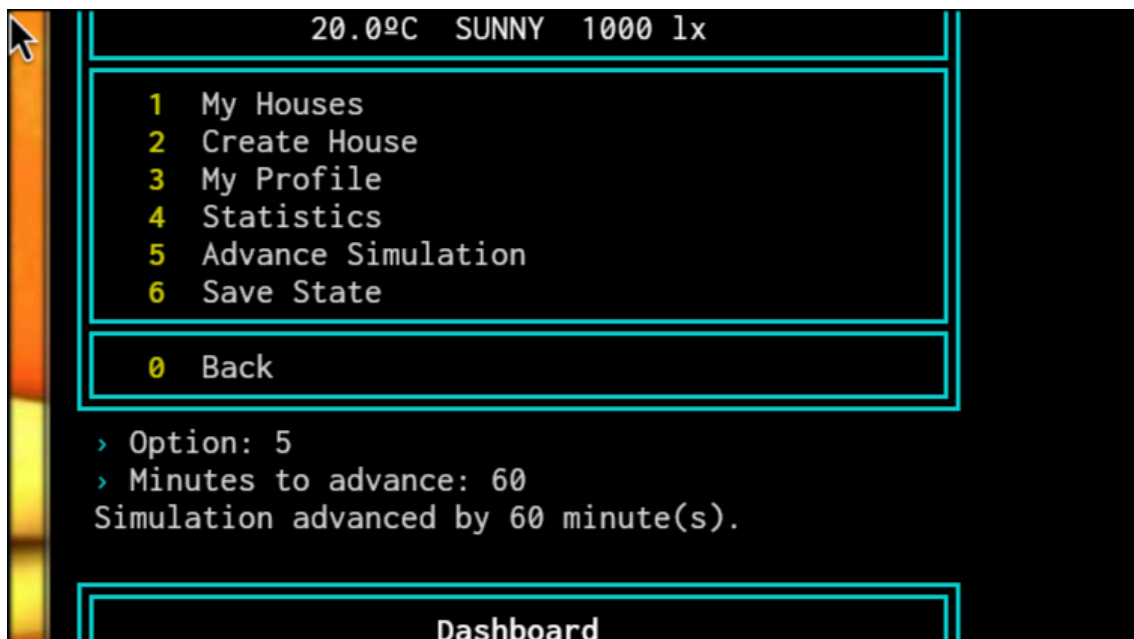
5.4. Menu Principal do Utilizador



5.4.1. Criar uma Casa

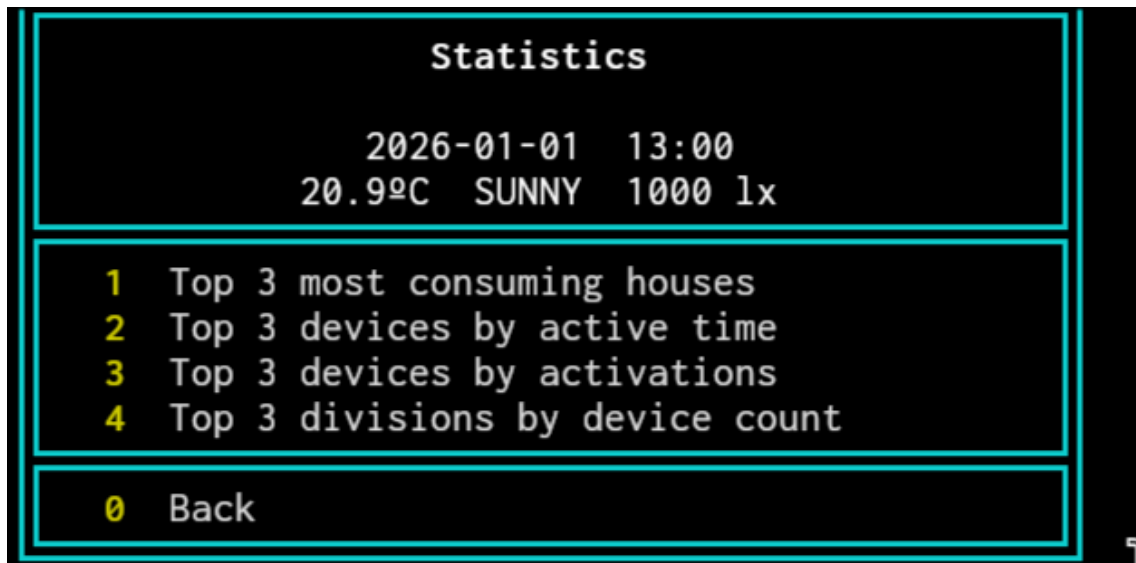


5.4.2. Avançar o Tempo



Quando automações disparam, os seus nomes são listados para que o utilizador saiba o que aconteceu no sistema durante o avanço.

5.4.3. Estatísticas



5.5. Menu de uma Casa



5.5.1. Adicionar Dispositivo

Device Type

2026-01-01 12:00
20.0°C SUNNY 1000 lx

1 Lamp

2 Speaker

3 Curtain

4 Gate

5 Plug

6 Relay

7 Heater

8 Fan

9 Air Conditioner

10 Television

11 Temperature Sensor

12 Luminosity Sensor

13 Rainfall Sensor

0 Back

> Option: 9

> Brand: daewood

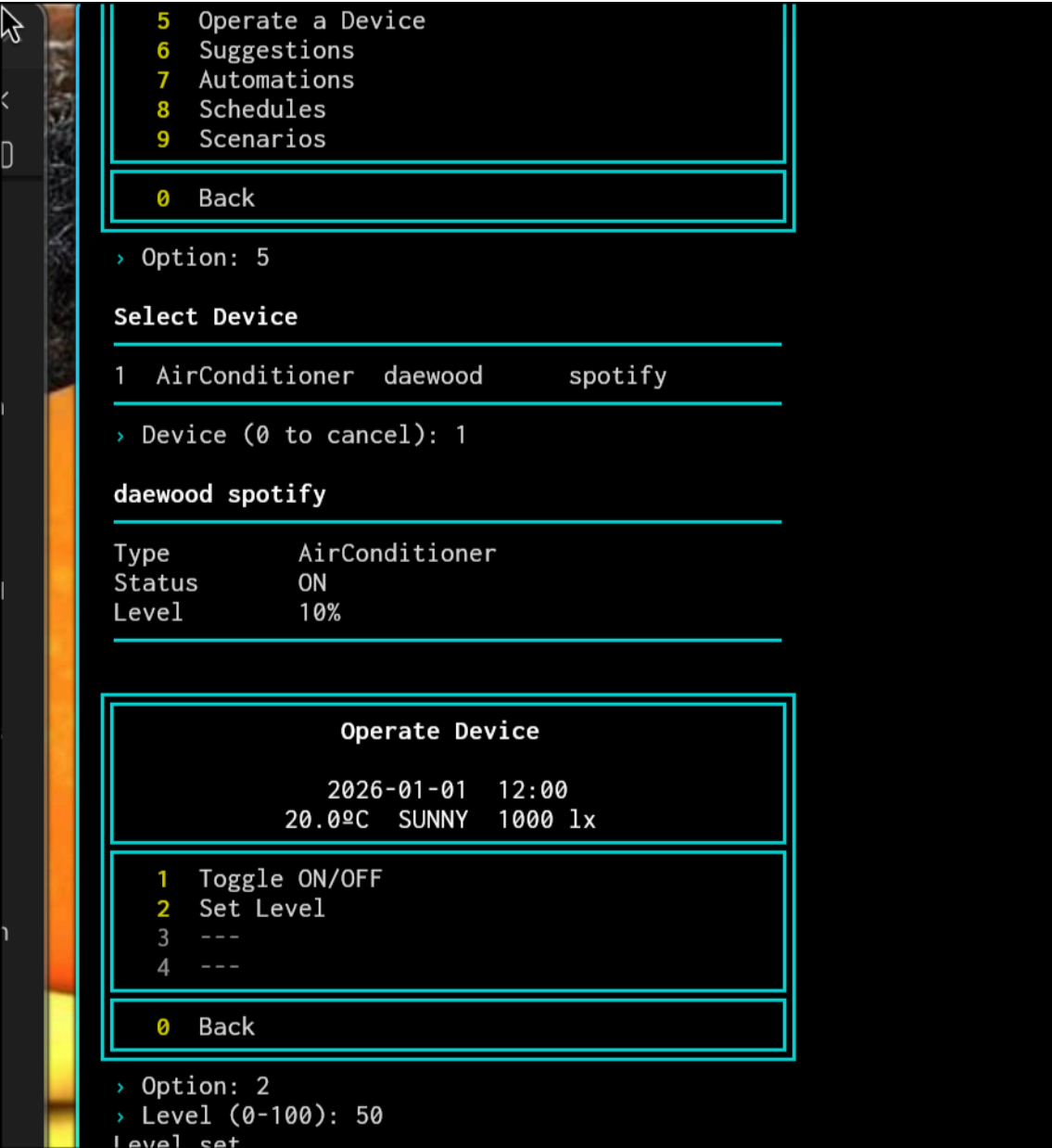
> Model name: spotify

> Consumption per hour (Wh/h): 10

> Cooling power (0-100): 10

Air conditioner added.

5.5.2. Operar um Dispositivo



5.5.3. Criar uma Automação

Automations

2026-01-01 12:00
20.0°C SUNNY 1000 lx

1 ---
2 Add Automation
3 ---

0 Back

> Option: 2
> Automation name: turnonaircondition

Available Devices

1 AirConditioner daewood spotify

> Device (0 to finish): 1

Action: spotify

2026-01-01 12:00
20.0°C SUNNY 1000 lx

1 Turn On
2 Turn Off
3 Set Level
4 ---
5 ---

0 Back

> Option: 1

5.5.4. Ver Sugestões

Analysing interaction history...

Suggestion 1 (Schedule):

"Lamp 'Hue White Ambiance' was manually turned on
at least 3 times around 08:02."

Accept? (y/n): y

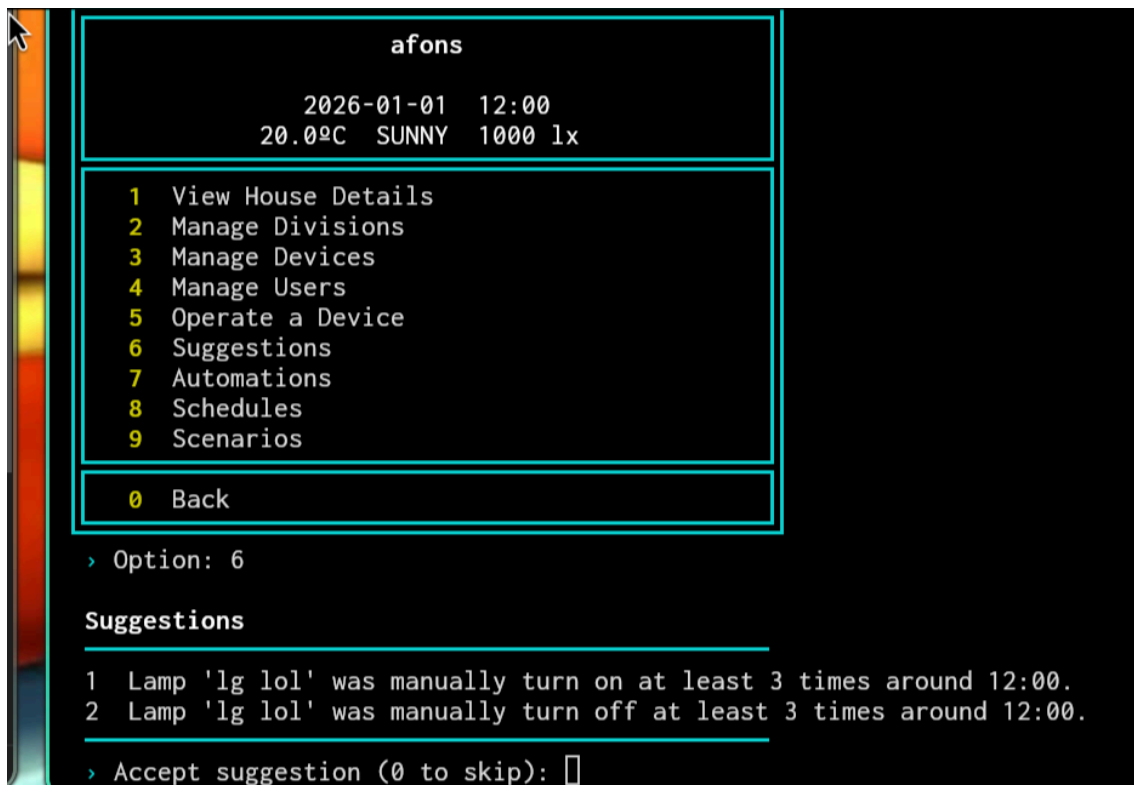
Schedule added.

Suggestion 2 (Automation):

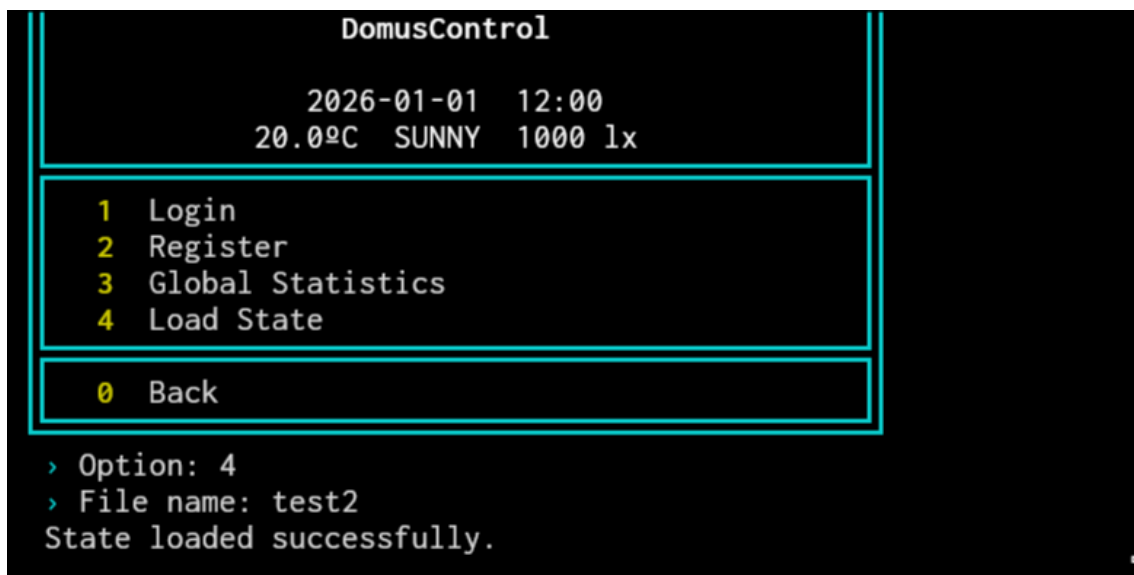
"Every time 'Relay Entrada' is turned off,
'Coluna Cozinha' is then turned off. Detected 4 times."

Accept? (y/n): n

Skipped.



5.6. Gravar e Carregar Estado



Ao carregar, toda a estrutura de objectos é restaurada com fidelidade, incluindo o histórico de interações, as automações definidas, o estado dos sensores, e o instante de simulação no momento em que foi gravado.

6. Testes

Foram desenvolvidos testes com o framework JUnit 5 de forma a cobrir de forma abrangente os diferentes módulos do sistema. São testadas as funcionalidades core da classe

DomusControl, nomeadamente o registo e autenticação de utilizadores, a gestão de casas e divisões, e o controlo de dispositivos. Os dispositivos (lâmpadas, ar condicionado, estores, sensores, entre outros) dispõem de testes unitários individuais que verificam os seus comportamentos específicos, como ajuste de nível, abertura e cor. O módulo de rotinas é igualmente coberto, com testes às automações, cenários, condições (temporais, de sensor e de estado) e ações associadas. Para além disso, estão presentes testes ao motor de sugestões, que valida a deteção de padrões de interação do utilizador, e ao módulo de simulação, que verifica o comportamento do estado ambiental ao longo do tempo. As exceções personalizadas do sistema também são testadas individualmente, garantindo que as mensagens de erro e o comportamento em situações inválidas são os esperados.

7. Conclusão

O desenvolvimento do DomusControl permitiu-nos cumprir todos os requisitos propostos no enunciado, incluindo a funcionalidade mais avançada: a geração automática de sugestões baseada no histórico de utilização.

O nosso foco principal foi desenhar uma arquitetura limpa, organizada e extensível. Ao aplicar conceitos de Programação Orientada a Objetos de forma prática, conseguimos separar bem as responsabilidades de cada parte do sistema. A divisão da interface gráfica em vários menus (`UserUI`, `HouseUI`, `ActionsUI`) evitou a criação de código confuso e difícil de manter. Ao mesmo tempo, a forma como estruturámos a hierarquia de dispositivos garante que adicionar novos equipamentos ou sensores no futuro seja um processo simples e direto.

A introdução de um motor de simulação que reage à passagem do tempo e às condições meteorológicas, aliada a um sistema de permissões por utilizador, tornou o projeto muito próximo de um sistema real de domótica.

Em suma, este projeto revelou-se um excelente desafio. Permitiu-nos consolidar os conhecimentos de Java e de desenho de software numa aplicação de grande escala, resultando num sistema final estável, seguro e pronto a ser expandido.